Lab Assignment

International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

1. Purpose of the Lab Assignment

The purpose of this lab assignment is to investigate digital evidence from a USB disk image and network traffic. It focuses on recovering deleted files and examining media for possible hidden content. The assignment also involves analysing FTP and HTTP traffic to identify file transfers. Recovered files and network objects are checked using file metadata and SHA-256 hashes. The aim is to produce a reliable forensic report based on the available evidence.

2. Lab Report Structure

The report begins with the case reference and student identification. It explains how the evidence was preserved and how the investigation was carried out. The findings cover disk-image examination, file recovery, steganography, FTP, and HTTP analysis. A UTC timeline and evidence log are included to support the investigation.

2.1. Title Page

• Lab Title: Case Study 4: Investigating Rhion Possession Evidence

• Student Name: Olanrewaju Wale Mustapha Oladimeji.

• Student ID: 2025/FWSD/11320.

• Course Name: SBT-DF204: Computer Forensic Case Study

• Instructor Name: Aminu Idris

• Date of Submission: 25/10/2026

• Version:

2.2. Executive Summary

This case study investigates supplied digital evidence to understand possible file possession, concealment, and transfer activity. The investigation involves examining a USB disk image and recovering relevant files. FTP and HTTP traffic are analysed to identify recorded network activity and reconstruct available objects. File types, metadata, timestamps, and SHA-256 hashes are used to validate the findings.

2.3. Lab Objectives

The first objective is to preserve the original evidence and verify its integrity using SHA-256 hashes. The second objective is to examine the USB disk image and recover relevant deleted or recoverable files. The third objective is to analyse recovered media for possible hidden content using authorised forensic tools. The fourth objective is to investigate FTP and HTTP traffic and validate any reconstructed files or archives. The final objective is to correlate the findings in a UTC timeline and produce a defensible forensic report

2.4. Tools and Resources Used

Kali Linux is used as the main environment for conducting the forensic investigation.
Sleuth Kit tools, including mmls, fsstat, fls, and icat, are used to examine the disk image and file-system records.
PhotoRec may be used to recover files.
ExifTool and authorised course tools support media and steganography analysis.
Wireshark and tshark are used to examine FTP and HTTP traffic and reconstruct available network objects.
Linux commands like file, stat, cp, and sha256sum are used to preserve evidence, record metadata, and verify file integrity.

2.5. Methodology

This individual case study brings together disk-image recovery, steganography, FTP, HTTP, and Wireshark analysis. You will work from the authorised course evidence package and produce an evidence-led account of what the artefacts do and do not establish. Your report must clearly separate direct observations from inferences about file transfer, concealment, possession, or the person who used a device.

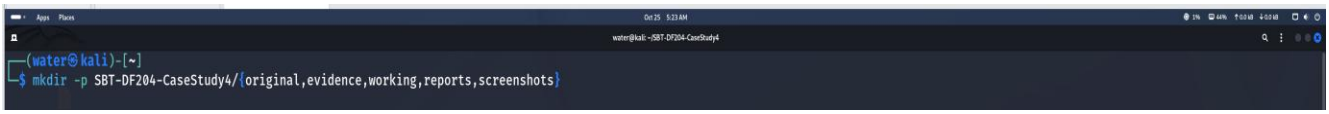
Case scenario

This is a fictional training investigation concerning imagery that may meet an illegal-possession threshold in the case scenario. The evidence package includes a seized USB disk image and network traces associated with FTP and HTTP activity. Investigators need to know whether deleted files, hidden data, reconstructed network objects, and the timing of recorded transfers form a defensible sequence. The task is an offline examination of supplied training artefacts; it does not authorise any contact with people, accounts, systems, hosts, or services named in the material.

Case materials

Use only the materials supplied through E-Campus and the accompanying teaching decks. The course sequence includes a USB DD image, an FTP trace (rhino.log), HTTP traces (rhino2.log and rhino3.log), and supporting recovery and Wireshark guidance. Use the exact filename in your evidence package if it differs from the deck. Before analysis, create a working copy and record integrity information for every original file. Do not alter, shred, or dispose of original course material before your work has been assessed.

Creating a File Folder for Case Study 4







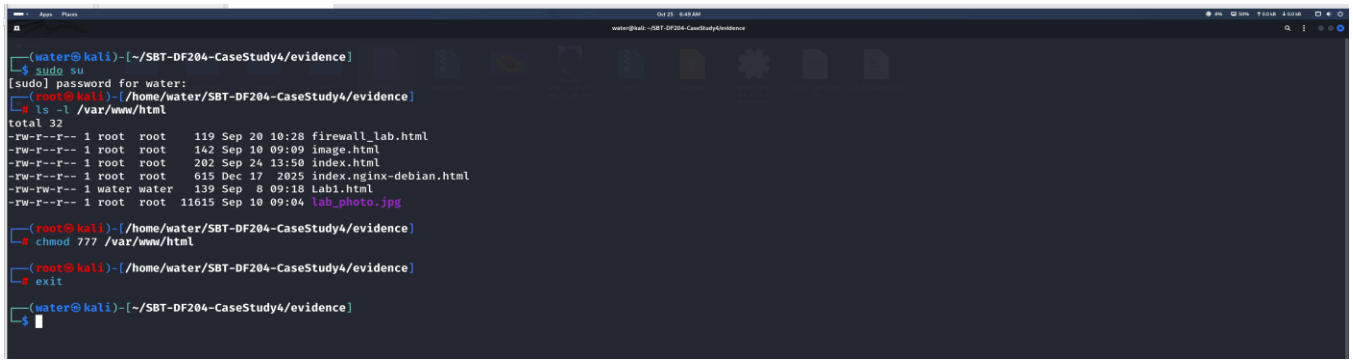
bd898cf94c5bcd7f94d8d4c61e65000c4ac1bb21265573411a0d2bc7e009143a
1tiPoLzElH1owRVyIBBoIgNs3Yvfqk2Hh?usp=sharing

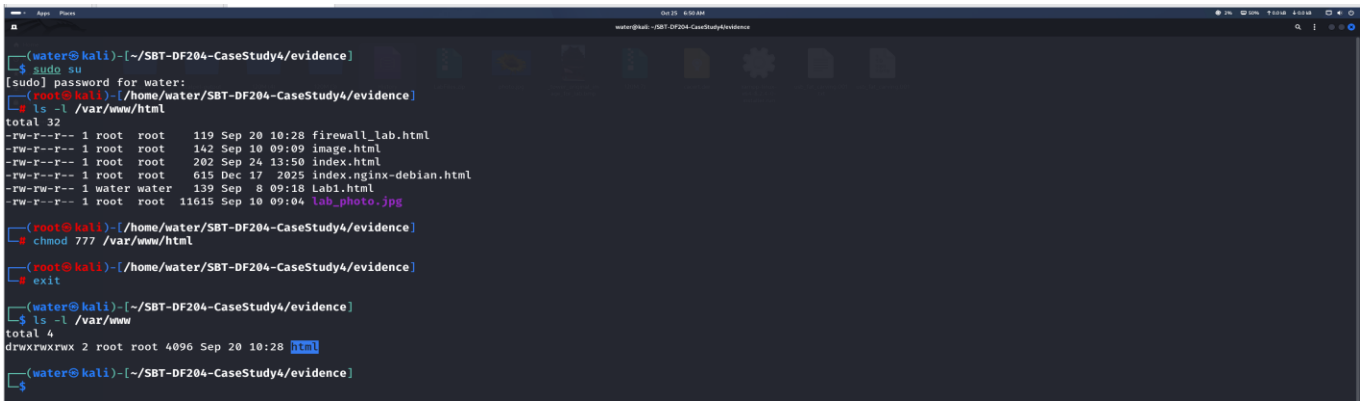
Downloading Basic Log file



81a22f991c85047afa3f9e53110b38631f9b8a75f73d820d266643510534daf7
Illegal_Possession_Images

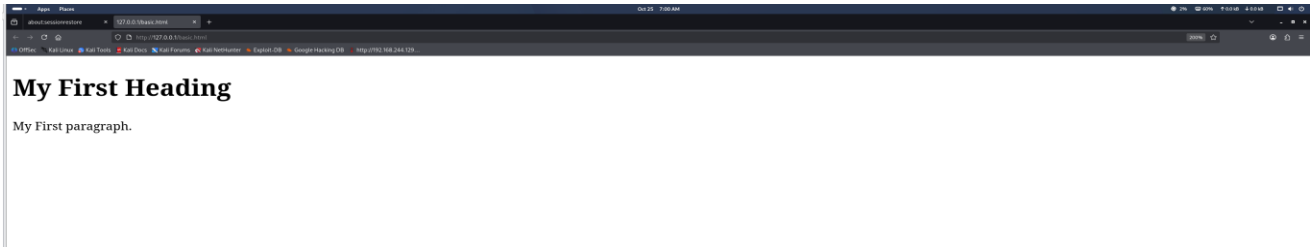
Create a simple html page to be observed in Wireshark





Create a simple webpage, named basic.html, Put it in the folder /var/www/html, Start web service, Show the webpage





My First Heading

My First paragraph.

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ curl -v 127.0.0.1/Lab1.html
*   Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 38482
* using HTTP/1.x
> GET /Lab1.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 25 Oct 2026 11:06:26 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 13:18:31 GMT
< ETag: "8b-65af8923728ee"
< Accept-Ranges: bytes
< Content-Length: 139
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html>
<body>
<h1>SBT-DF203 HTTP Evidence</h1>
<p>Name: Olanrewaju Wale</p>
<p>Reg No: 2025/FWSD/11320</p>
</body>
</html>
* Connection #0 to host 127.0.0.1:80 left intact

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$
```

Downloading Basic.log file

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ cd ~/SBT-DF204-CaseStudy4/evidence
wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/basic.log
--2026-10-25 07:17:47--  https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/basic.log
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1812 (1.8K) [application/octet-stream]
Saving to: 'basic.log'

basic.log                                           100%[=============================================================================================================================>]   1.77K  --.-KB/s    in 0s

2026-10-25 07:17:48 (5.79 MB/s) - 'basic.log' saved [1812/1812]


┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$
```

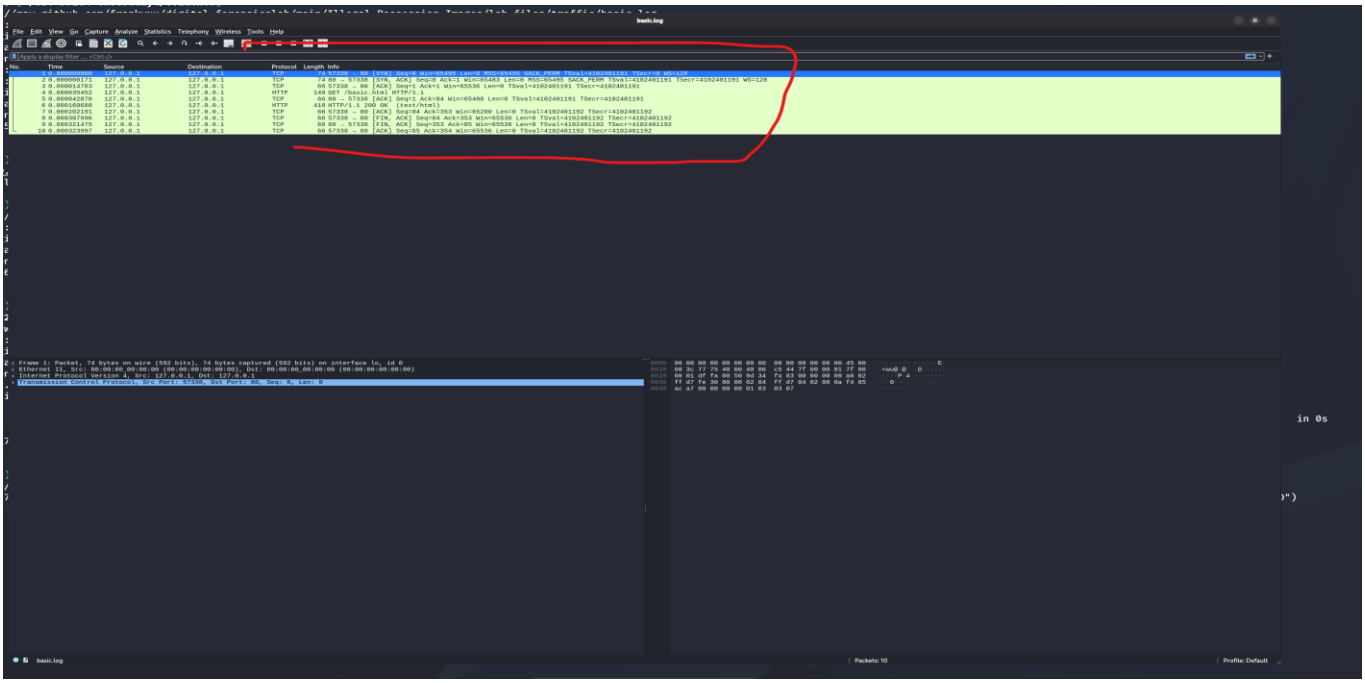
```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ sha256sum basic.log
7918be717c9c820da1921eea6dfb24e461aff471740b90e062faabc2a3494b5f  basic.log

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$
```

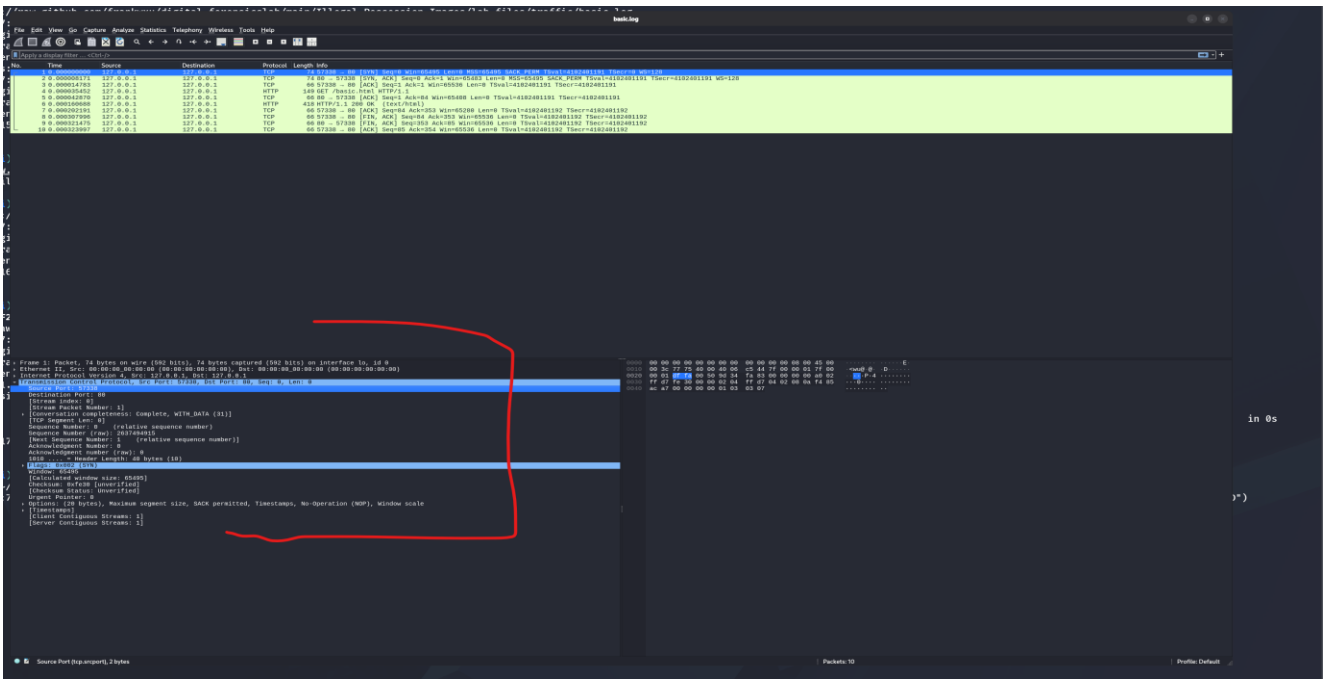
sha256sum basic.log

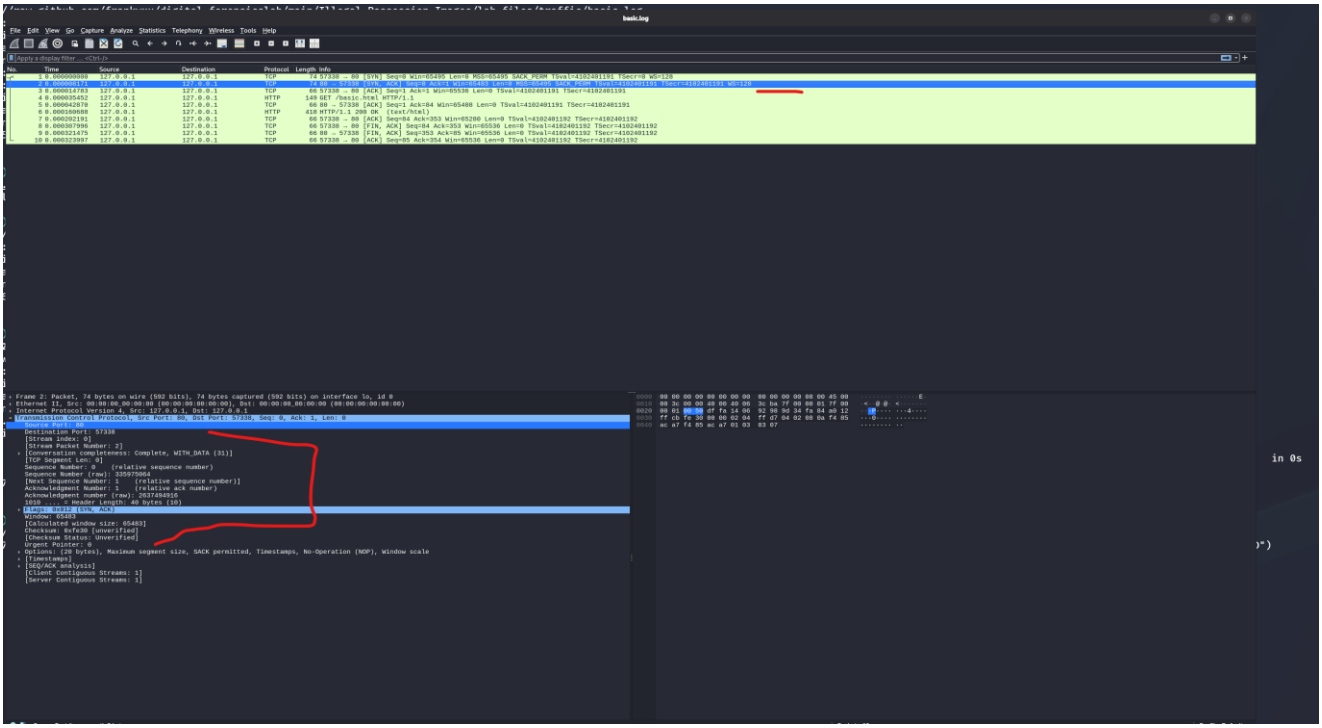
7918be717c9c820da1921eea6dfb24e461aff471740b90e062faabc2a3494b5f basic.log

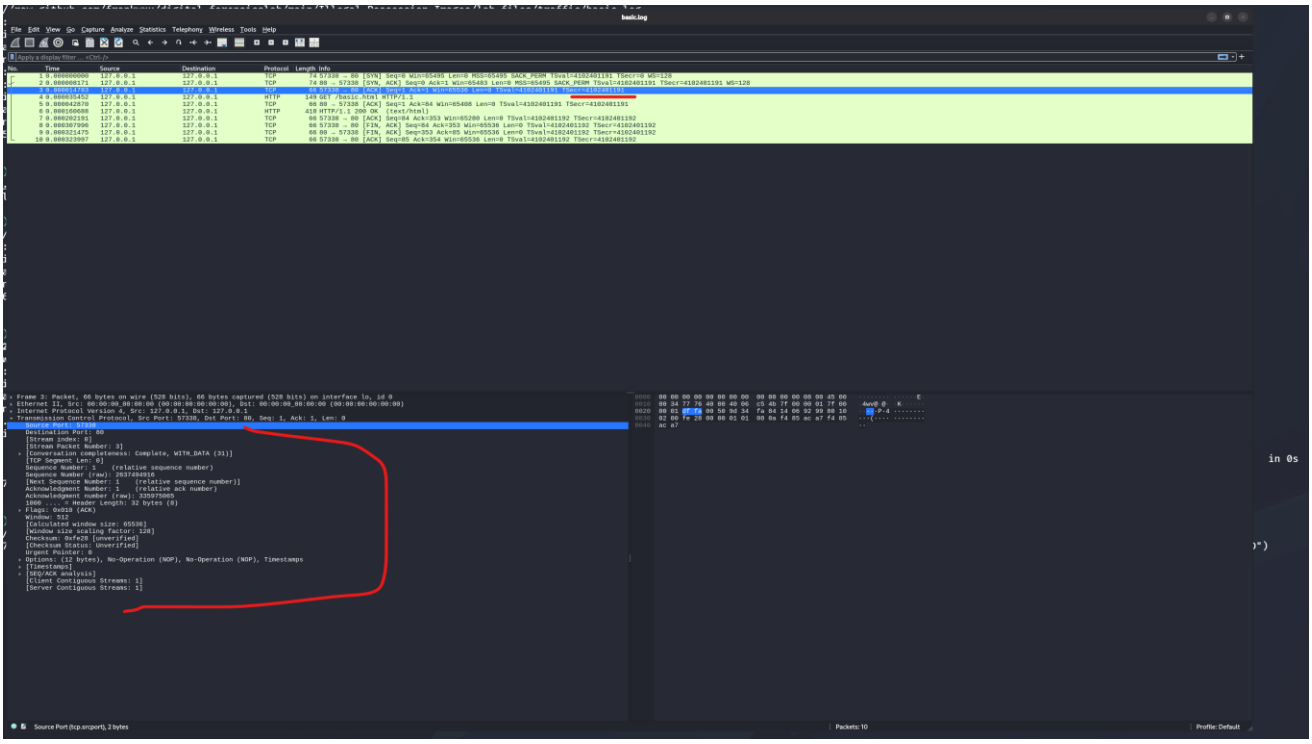
Capture the handshake,http response, request and tears down



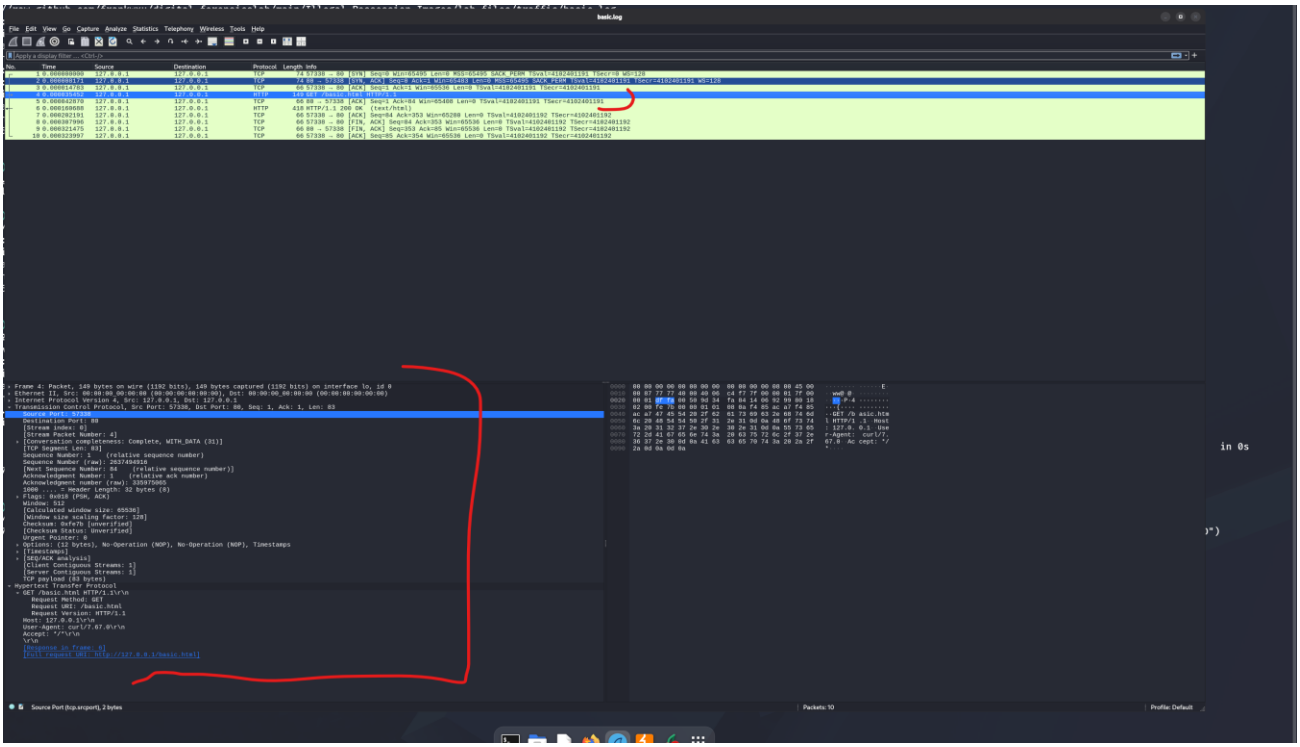
Hand shake



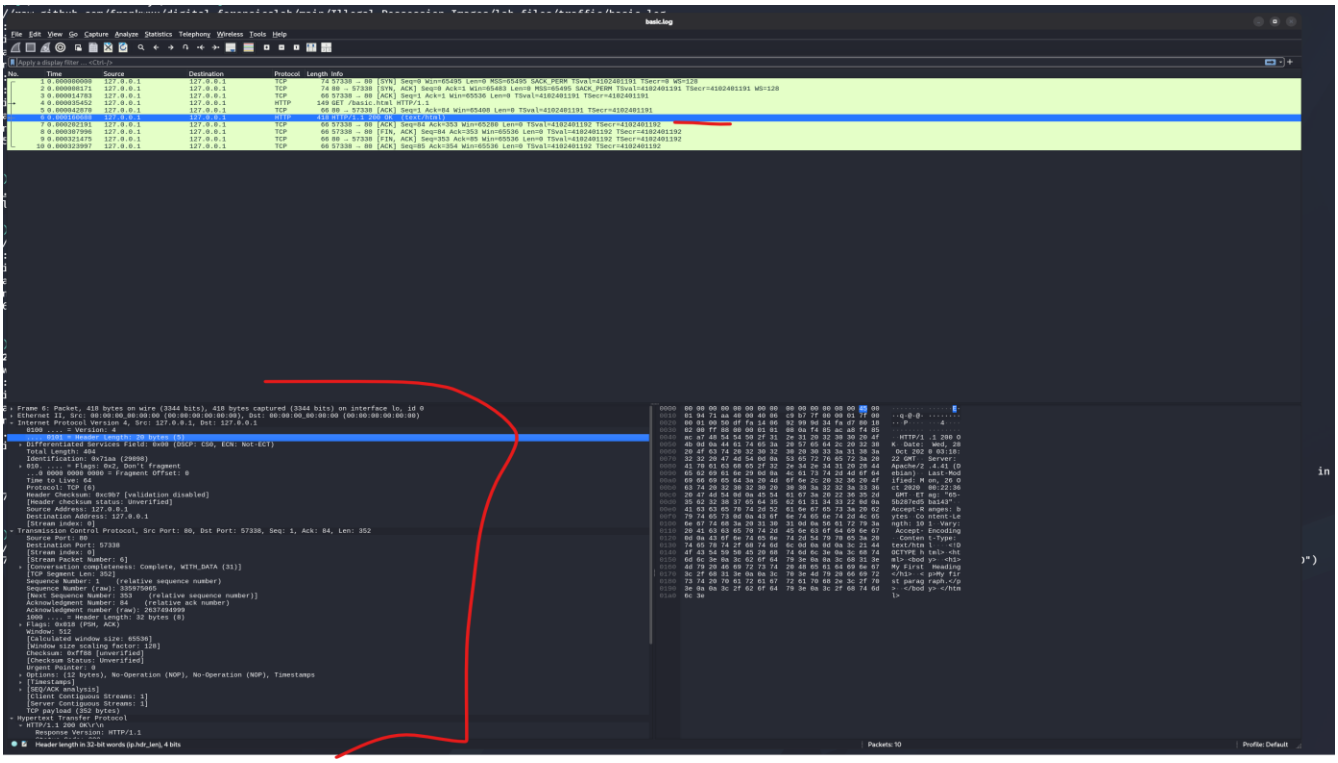




HTTP Request (Get Request)



HTTP Response



Downloading Image2.log

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/image2.log
--2026-10-25 08:16:24--  https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/image2.log
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.111.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 73692 (72K) [application/octet-stream]
Saving to: 'image2.log'

image2.log                                          100%[=============================================================================================================================>]  71.96K   418KB/s    in 0.2s

2026-10-25 08:16:25 (418 KB/s) - 'image2.log' saved [73692/73692]


┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ ls
```

Sha256sum

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ ls
'1tiPoLzElH1owRVyIBBoIgNs3Yvfqk2Hh?usp=sharing'   basic.log   Illegal_Possession_Images   image2.log

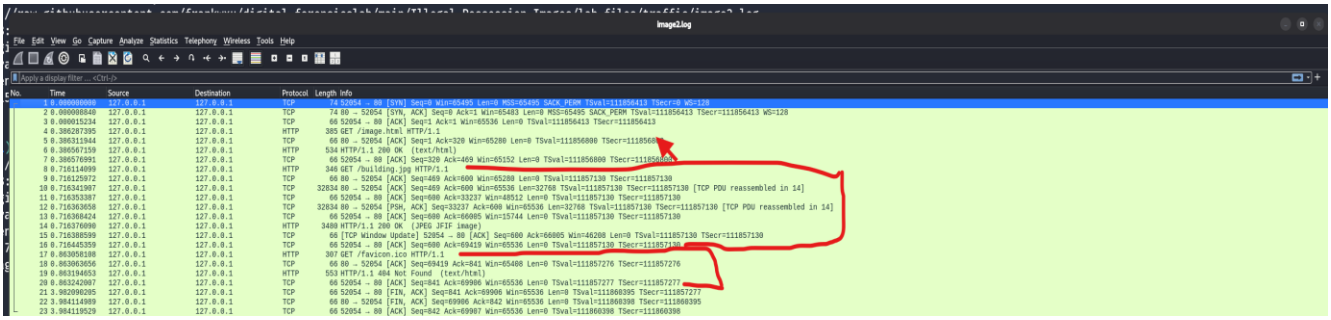
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ sha256sum image2.log
4d9bc50bc05213828ff7807f8bd6bb2b40f5f8da127c20936b774086187dd594  image2.log

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$
```

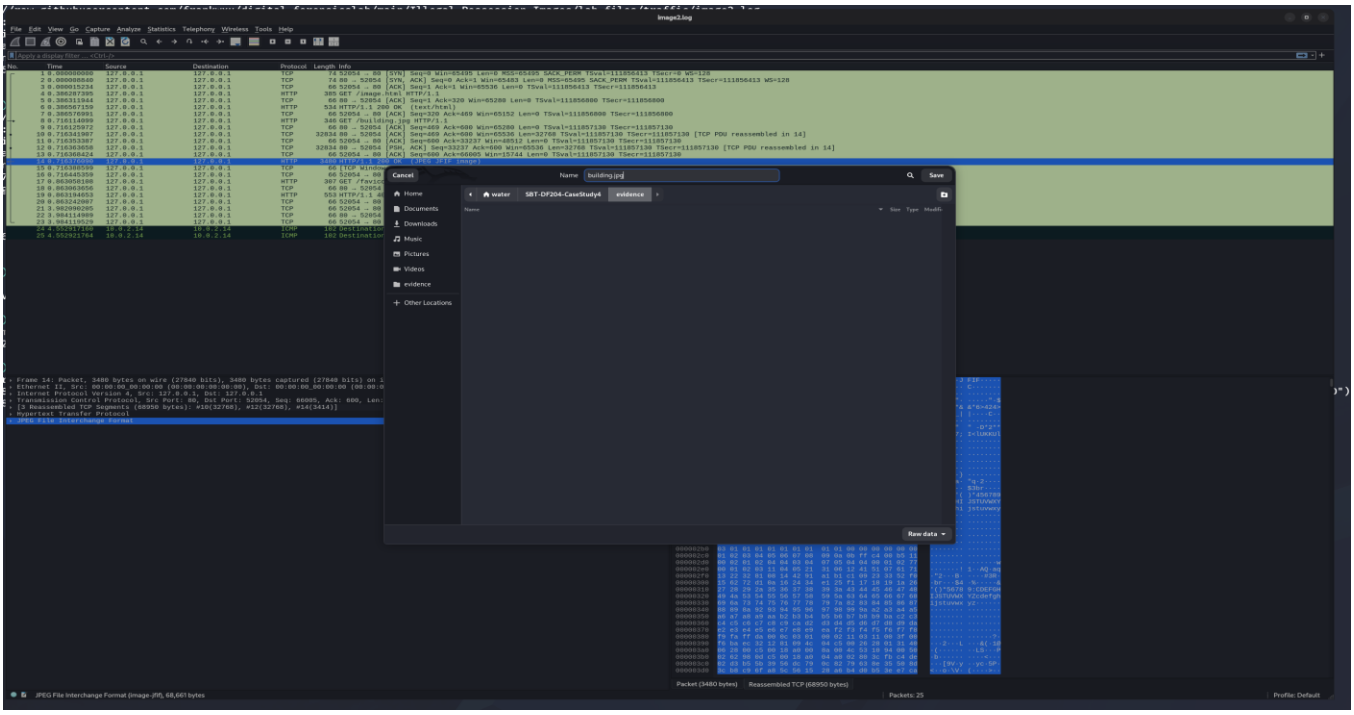
sha256sum image2.log

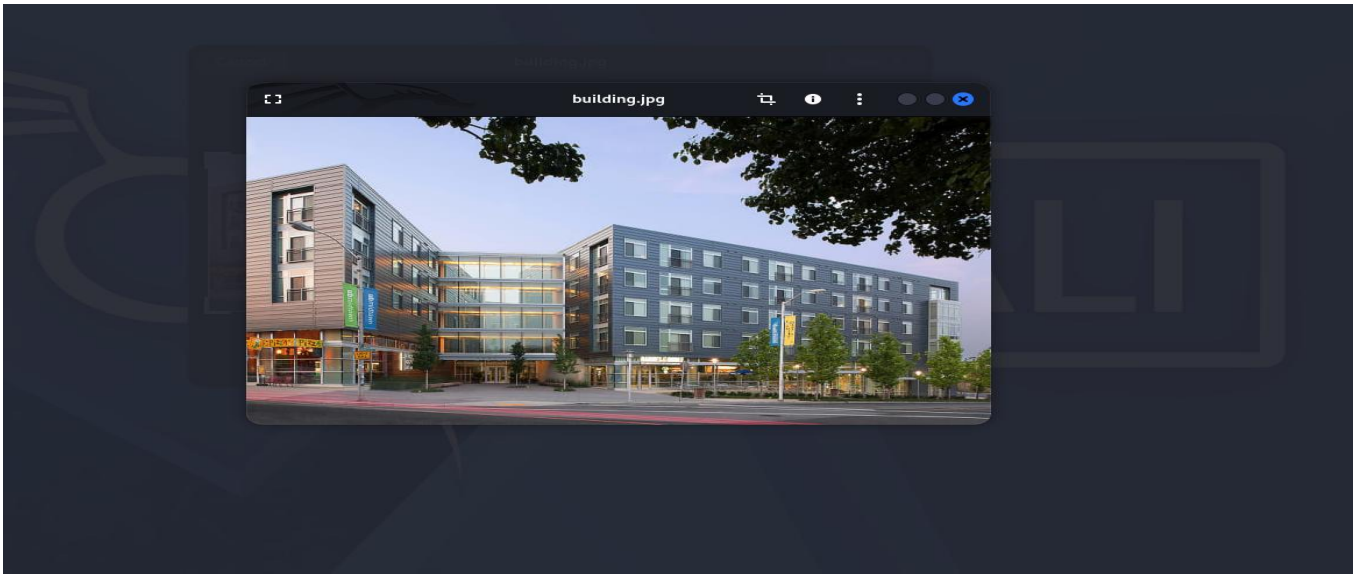
4d9bc50bc05213828ff7807f8bd6bb2b40f5f8da127c20936b774086187dd594 image2.log

First http Get (html), Second http Get(JPG), Third http Get(ico)



Extract the building.jpg from traffic






```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$ sha256sum building.jpg
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeeef97afdb  building.jpg

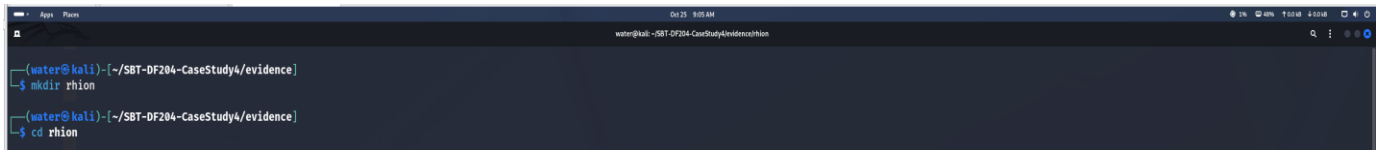
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence]
└─$
```

sha256sum building.jpg

a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeeef97afdb building.jpg

Rhino Hunting – Illegal Possession Investigation

Create a working folder



Download DD image and unzip it



Verify MD5 hashes



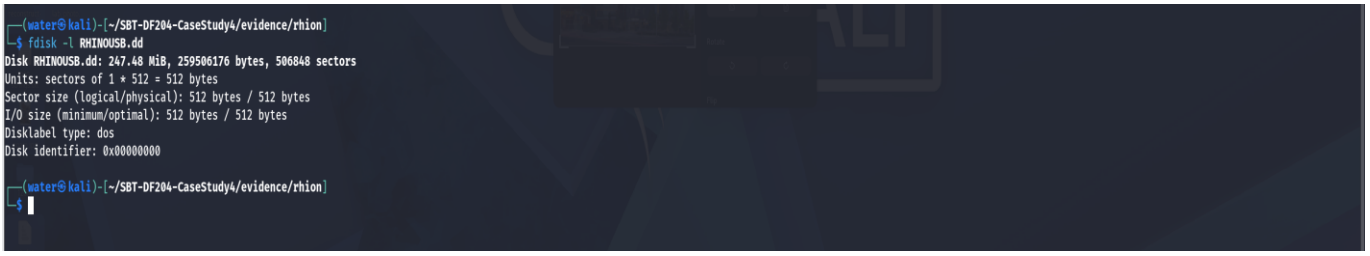
openssl dgst -md5 rhino.log MD5(rhino.log)= c0d0093eb1664cd7b73f3a5225ae3f30

openssl dgst -md5 rhino2.log MD5(rhino2.log)= cd21eaf4acfb50f71ffff857d7968341

openssl dgst -md5 rhino3.log MD5(rhino3.log)= 7e29f9d67346df25faaf18efcd95fc30

openssl dgst -md5 RHINOUSB.dd MD5(RHINOUSB.dd)= 80348c58eec4c328ef1f7709adc56a54

View partitions using fdisk



Displays the layout of a disk, including the unallocated spaces.



Shows file system details and statistics including layout, sizes, and labels



List file and directory names in a disk image

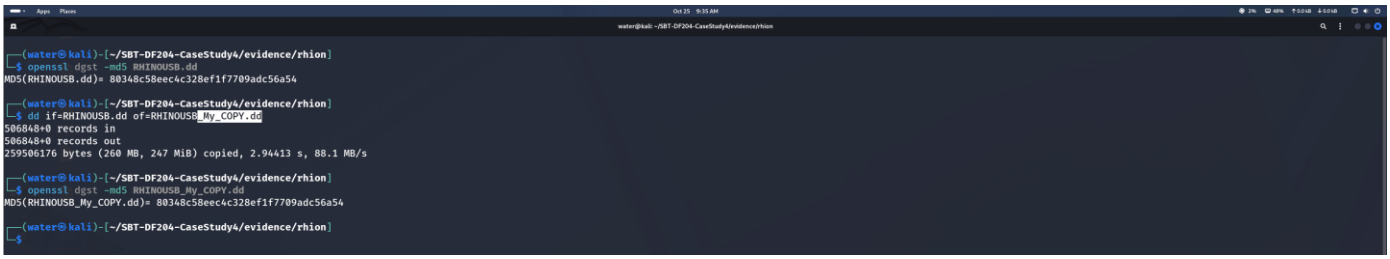


Extract the data units of a file, which is specified by its meta data address



Recover Rhino Images from DD

Make a copy of the DD avoid corrupt the original DD image

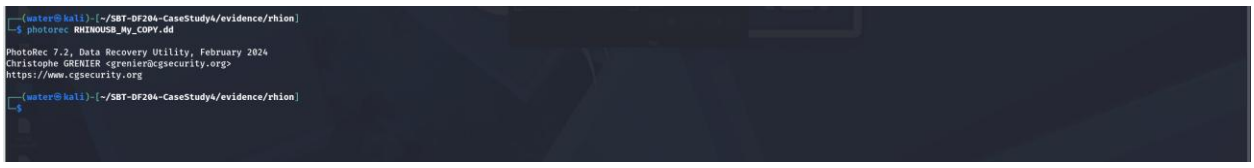


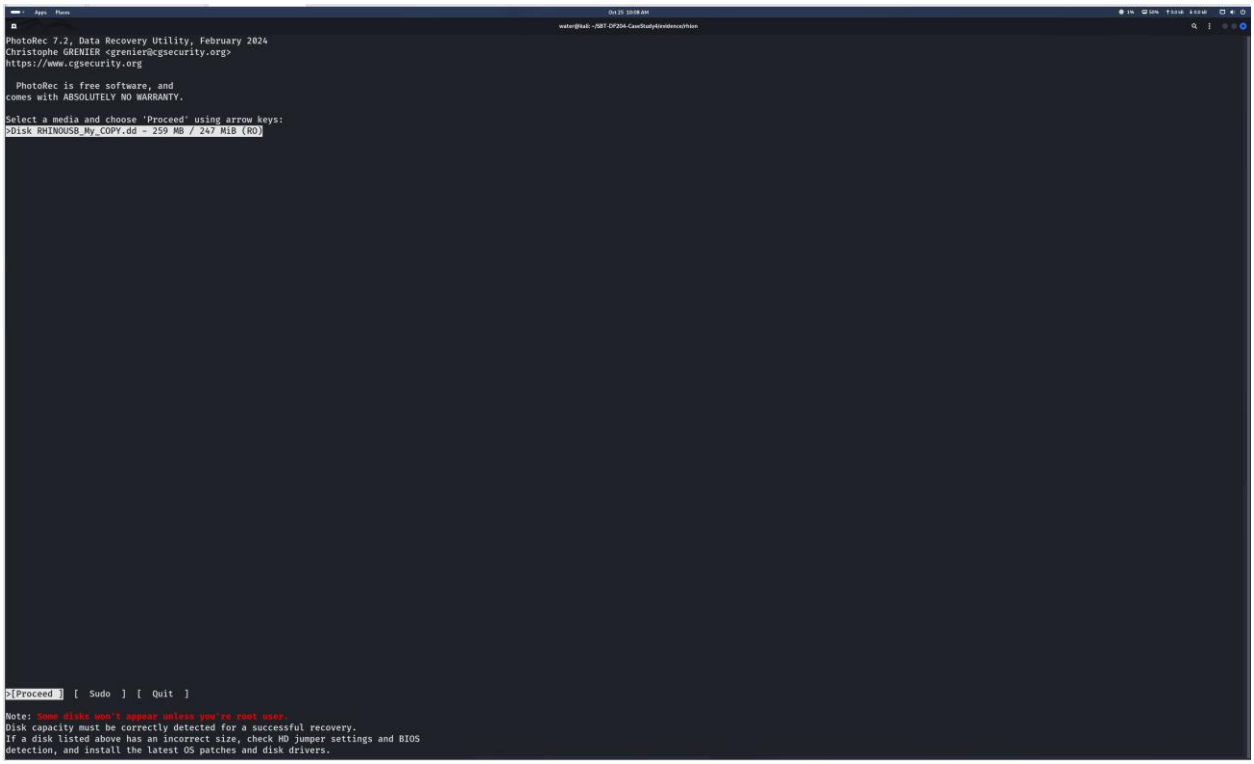
Verify both DD images are the same

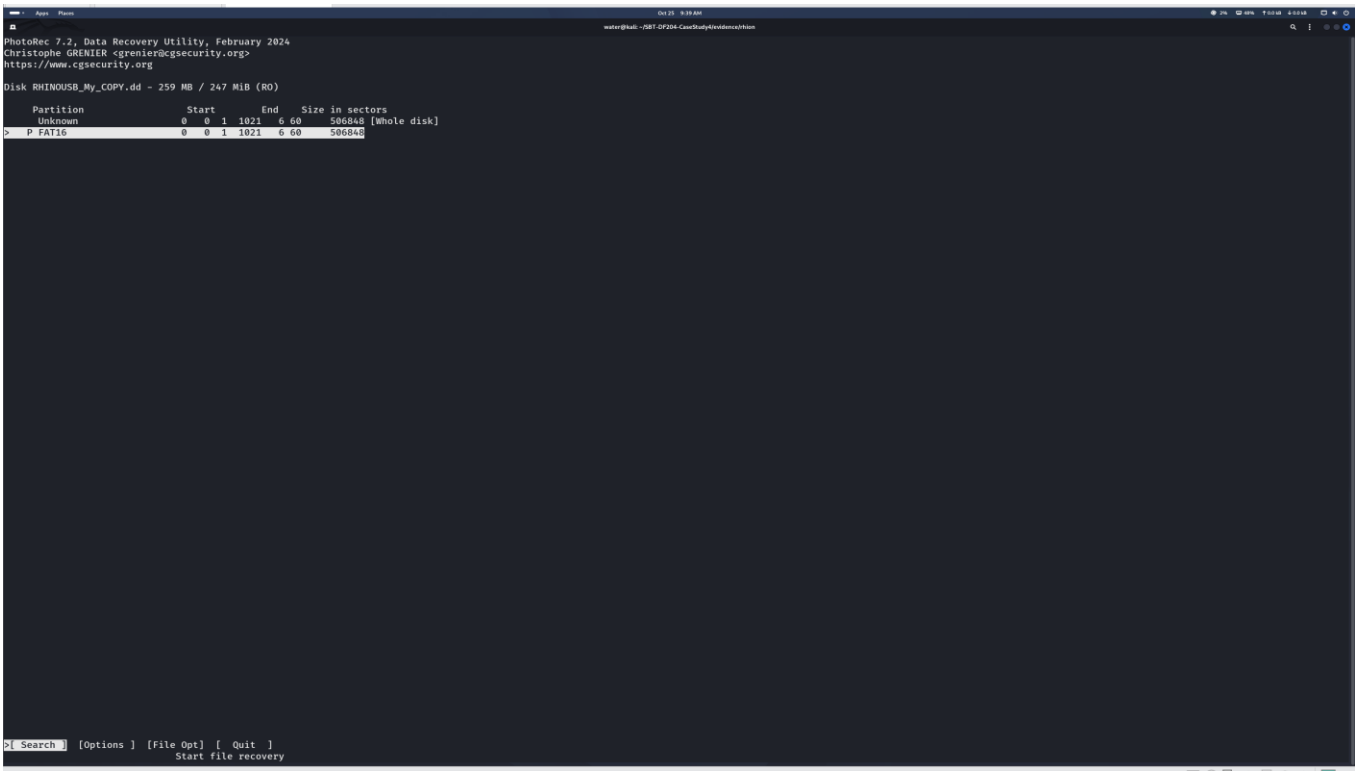
openssl dgst -md5 RHINOUSB.dd MD5(RHINOUSB.dd)= 80348c58eec4c328ef1f7709adc56a54

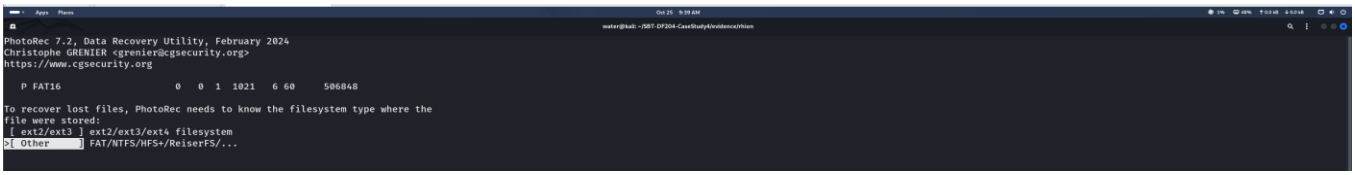
openssl dgst -md5 RHINOUSB_My_COPY.dd MD5(RHINOUSB_My_COPY.dd)= 80348c58eec4c328ef1f7709adc56a54

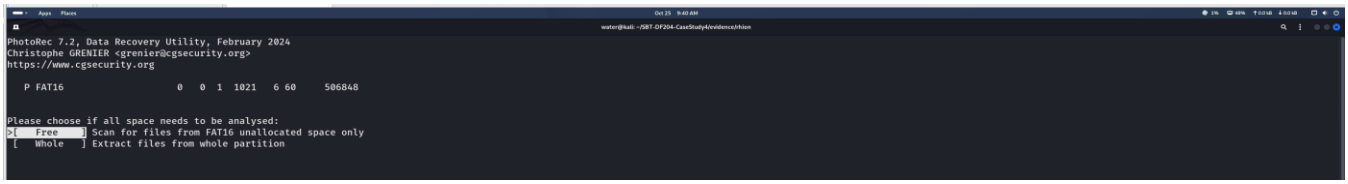
Start recover photos from our DD image

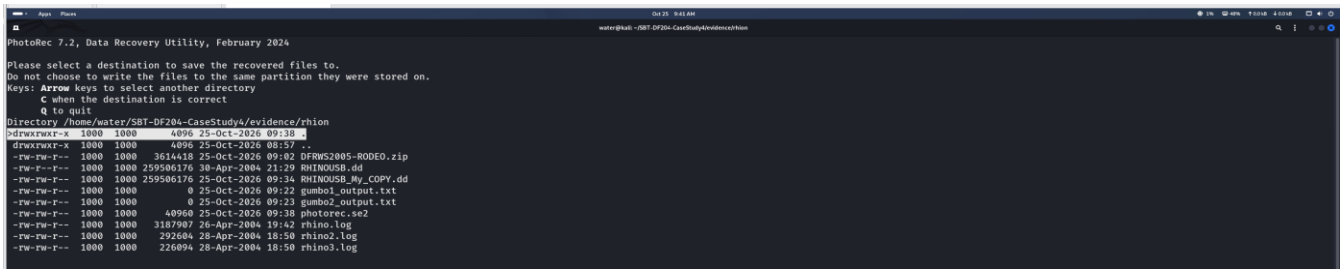


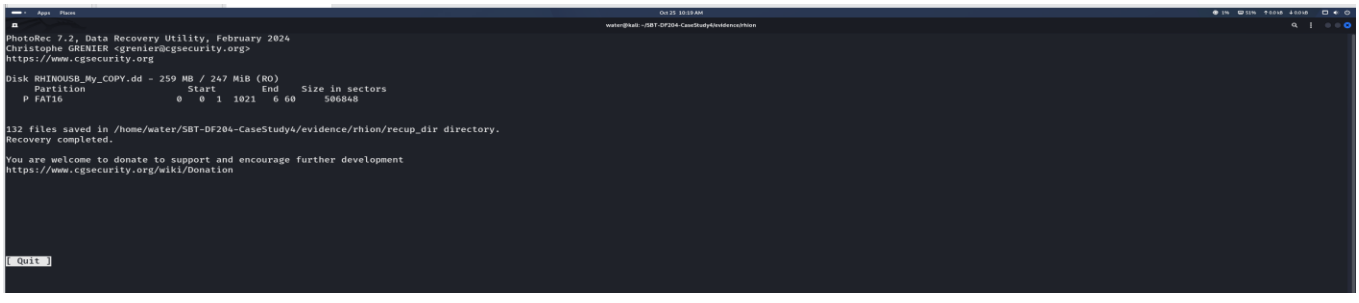



PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk RHINOUSB_My_COPY.dd - 259 MB / 247 MiB (RO)
Partition Start End Size in sectors
Unknown 0 0 1 1021 6 60 506848 [Whole disk]
> P FAT16 0 0 1 1021 6 60 506848
>[Search] [Options] [File Opt] [Quit]
Start file recovery


PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
P FAT16 0 0 1 1021 6 60 506848
To recover lost files, PhotoRec needs to know the filesystem type where the
file were stored:
[ext2/ext3] ext2/ext3/ext4 filesystem
>[Other] FAT/NTFS/HFS+/ReiserFS/...


PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
P FAT16 0 0 1 1021 6 60 506848
Please choose if all space needs to be analysed:
>[Free] Scan for files from FAT16 unallocated space only
[Whole] Extract files from whole partition


PhotoRec 7.2, Data Recovery Utility, February 2024
Please select a destination to save the recovered files to.
Do not choose to write the files to the same partition they were stored on.
Keys: Arrow keys to select another directory
C when the destination is correct
Q to quit
Directory /home/water/SBT-DF204-CaseStudy4/evidence/rhion
>drwxrwxr-x 1000 1000 4096 25-Oct-2026 09:38 .
drwxrwxr-x 1000 1000 4096 25-Oct-2026 08:57 ..
-rw-rw-r-- 1000 1000 3614418 25-Oct-2026 09:02 DFRWS2005-RODEO.zip
-rw-r--r-- 1000 1000 259506176 30-Apr-2004 21:29 RHINOUSB.dd
-rw-rw-r-- 1000 1000 259506176 25-Oct-2026 09:34 RHINOUSB_My_COPY.dd
-rw-rw-r-- 1000 1000 0 25-Oct-2026 09:22 gumbo1_output.txt
-rw-rw-r-- 1000 1000 0 25-Oct-2026 09:23 gumbo2_output.txt
-rw-rw-r-- 1000 1000 40960 25-Oct-2026 09:38 photorec.se2
-rw-rw-r-- 1000 1000 3187907 26-Apr-2004 19:42 rhino.log
-rw-rw-r-- 1000 1000 292604 28-Apr-2004 18:50 rhino2.log
-rw-rw-r-- 1000 1000 226094 28-Apr-2004 18:50 rhino3.log


PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
https://www.cgsecurity.org
Disk RHINOUSB_My_COPY.dd - 259 MB / 247 MiB (RO)
Partition Start End Size in sectors
P FAT16 0 0 1 1021 6 60 506848
132 files saved in /home/water/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir directory.
Recovery completed.
You are welcome to donate to support and encourage further development
https://www.cgsecurity.org/wiki/Donation
[Quit]

Verify if the folder exsits

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$ ls -l
total 514060
-rw-rw-r-- 1 water water   3614418 Oct 25 09:02 DFRWS2005-RODEO.zip
-rw-rw-r-- 1 water water         0 Oct 25 09:22 gumbo1_output.txt
-rw-rw-r-- 1 water water         0 Oct 25 09:23 gumbo2_output.txt
-rw-rw-r-- 1 water water     40960 Oct 25 10:08 photorec.se2
drwxrwxr-x 2 water water      4096 Oct 25 10:18 recup_dir.1
-rw-rw-r-- 1 water water    292604 Apr 28  2004 rhino2.log
-rw-rw-r-- 1 water water    226094 Apr 28  2004 rhino3.log
-rw-rw-r-- 1 water water   3187907 Apr 26  2004 rhino.log
-rw-r--r-- 1 water water 259506176 Apr 30  2004 RHINOUSB.dd
-rw-rw-r-- 1 water water 259506176 Oct 25 09:34 RHINOUSB_My_COPY.dd

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$
```

List all recovered files

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$ ls recup_dir.1/
f0000545.txt  f0335017_She_died_in_February_at_the_age_of_74.doc  f0339545.txt  f0344681.txt  f0349825.txt  f0354961.txt  f0360105.txt  f0365249.txt  f0370385.txt  f0375529.txt  f0380665.txt  f0385809.txt  f0390945.txt  f0396089.txt
f0104057.jpg  f0335081.jpg                                        f0340057.txt  f0345201.txt  f0350337.txt  f0355481.txt  f0360969.txt  f0365761.txt  f0370897.txt  f0376041.txt  f0381185.txt  f0386321.txt  f0391465.txt  f0396601.txt
f0104249.jpg  f0335601.txt                                        f0340569.txt  f0345713.txt  f0350849.txt  f0355993.txt  f0361137.txt  f0366273.txt  f0371417.txt  f0376553.txt  f0381697.txt  f0386833.txt  f0391977.txt  report.xml
f0105065.jpg  f0335945.txt                                        f0341081.txt  f0346225.txt  f0351369.txt  f0356505.txt  f0361649.txt  f0366785.txt  f0371929.txt  f0377353.txt  f0382209.txt  f0387353.txt  f0392489.txt
f0105873.jpg  f0336457.txt                                        f0341601.txt  f0346737.txt  f0351881.txt  f0357017.txt  f0362161.txt  f0367305.txt  f0372441.txt  f0377585.txt  f0382721.txt  f0387865.txt  f0393001.txt
f0106393.jpg  f0336969.txt                                        f0342113.txt  f0347257.txt  f0352393.txt  f0357537.txt  f0362673.txt  f0367817.txt  f0372953.txt  f0378097.txt  f0383241.txt  f0388377.txt  f0393737.txt
f0106409.jpg  f0337489.txt                                        f0342625.txt  f0347769.txt  f0352905.txt  f0358049.txt  f0363193.txt  f0368329.txt  f0373473.txt  f0378609.txt  f0383753.txt  f0388889.txt  f0394033.txt
f0106865.gif  f0338001.txt                                        f0343137.txt  f0348281.txt  f0353425.txt  f0358561.txt  f0363705.txt  f0368841.txt  f0373985.txt  f0379129.txt  f0384265.txt  f0389409.txt  f0394545.txt
f0106889.gif  f0338513.txt                                        f0343657.txt  f0348793.txt  f0353937.txt  f0359073.txt  f0364217.txt  f0369361.txt  f0374497.txt  f0379641.txt  f0384777.txt  f0389921.txt  f0395065.txt
f0106905.txt  f0339025.txt                                        f0344585.txt  f0349313.txt  f0354449.txt  f0359593.txt  f0364729.txt  f0369873.txt  f0375009.txt  f0380153.txt  f0385297.txt  f0390433.txt  f0395577.txt

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$
```

Navigate to the folder where the recovered files were saved/List all .jpg

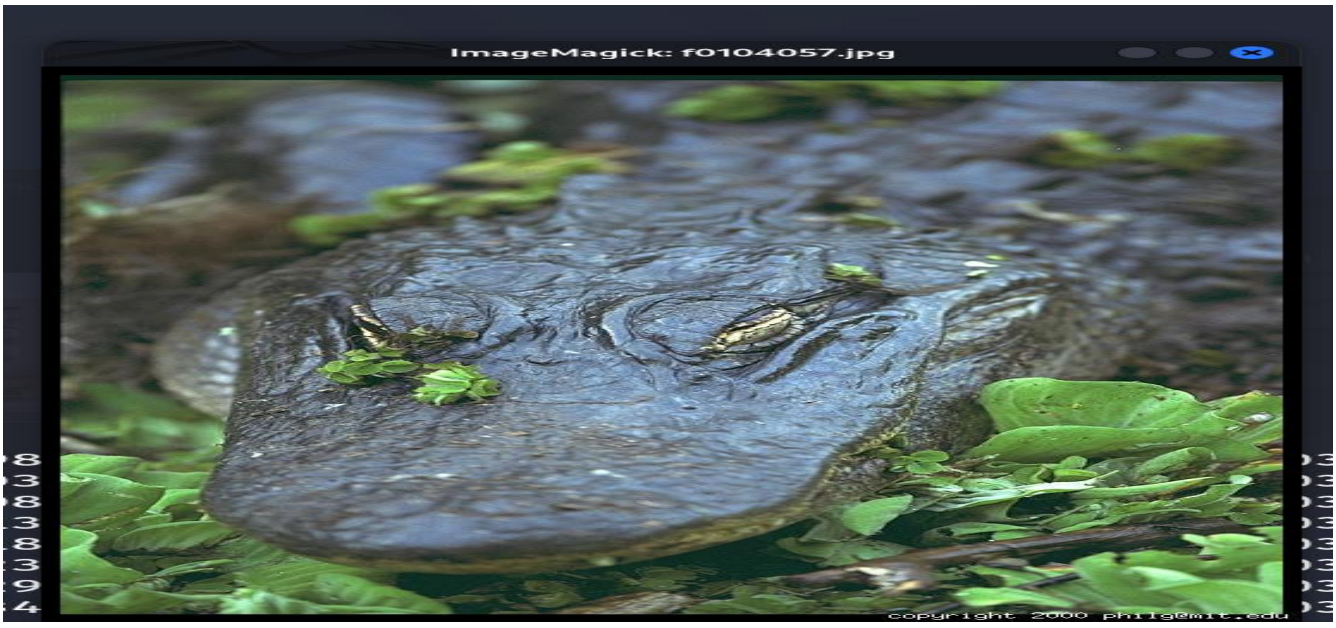
```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$ cd recup_dir.1/

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir.1]
└─$ ls -l *.jpg
-rw-rw-r-- 1 water water  95814 Oct 25 10:18 f0104057.jpg
-rw-rw-r-- 1 water water 415534 Oct 25 10:18 f0104249.jpg
-rw-rw-r-- 1 water water 411361 Oct 25 10:18 f0105065.jpg
-rw-rw-r-- 1 water water 264600 Oct 25 10:18 f0105873.jpg
-rw-rw-r-- 1 water water   6809 Oct 25 10:18 f0106393.jpg
-rw-rw-r-- 1 water water 230665 Oct 25 10:18 f0106409.jpg
-rw-rw-r-- 1 water water 264600 Oct 25 10:18 f0335081.jpg

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir.1]
└─$
```

Show one retrieved .jpg photo

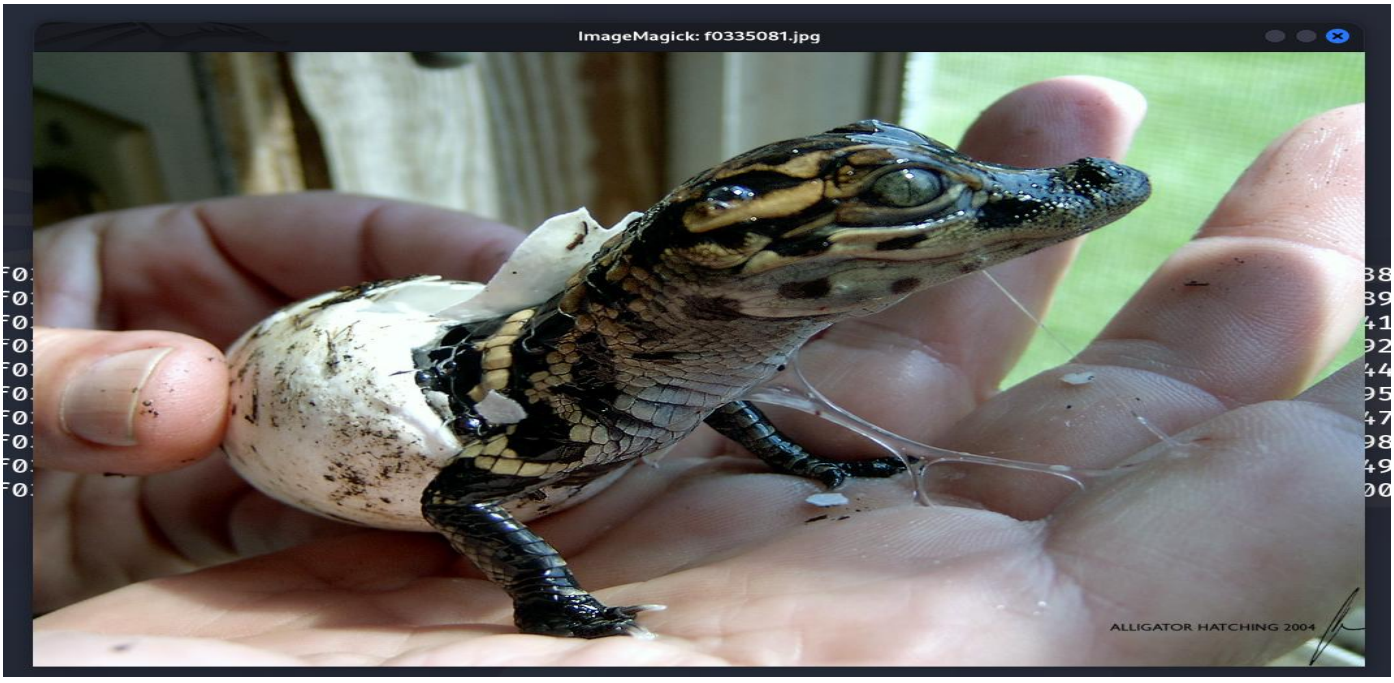
```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir.1]
└─$ display f0104057.jpg
```



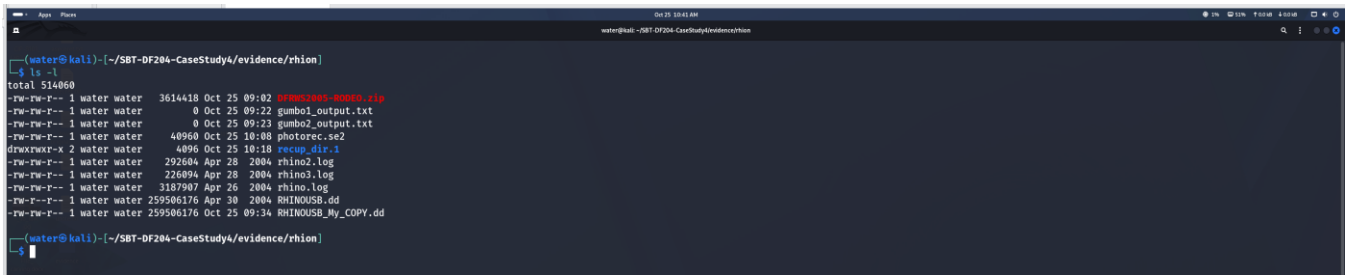
```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir.1]
└─$ display f0105065.jpg
```



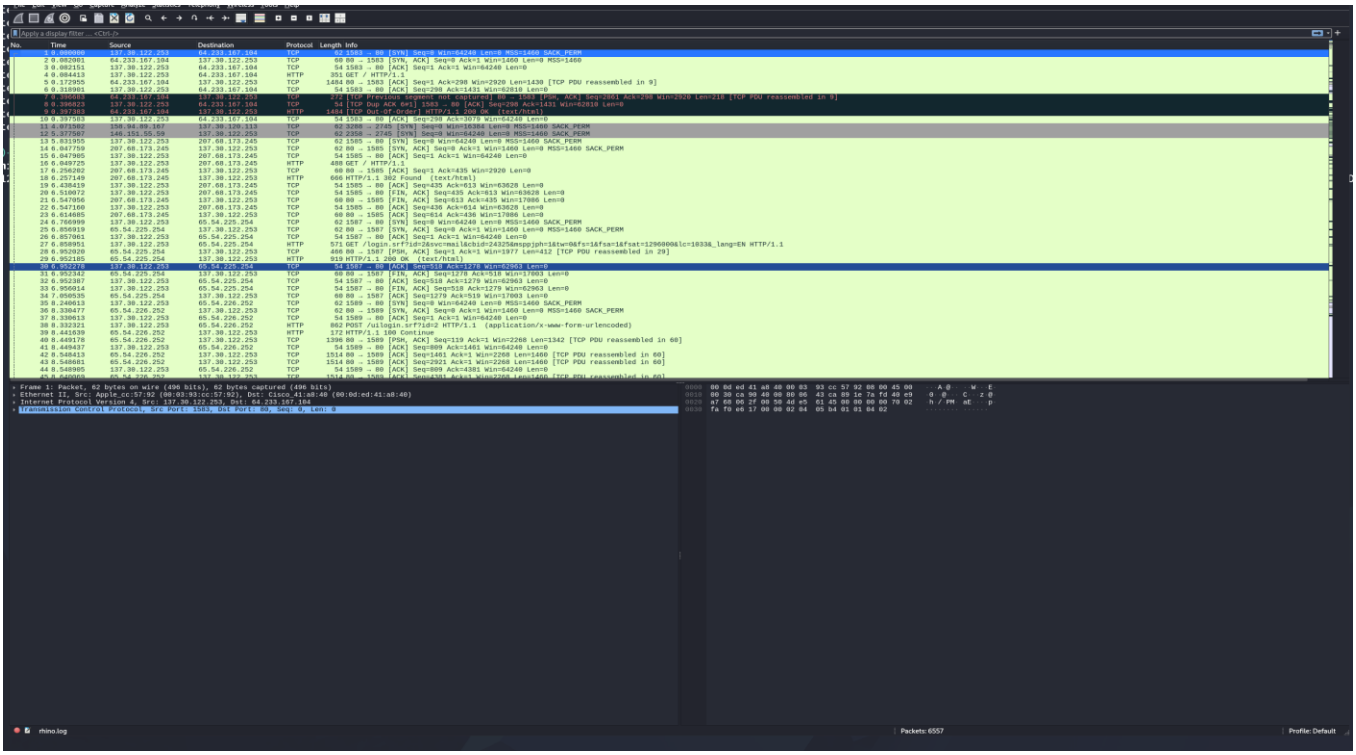

```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion/recup_dir.1]
└─$ display f0335081.jpg
```



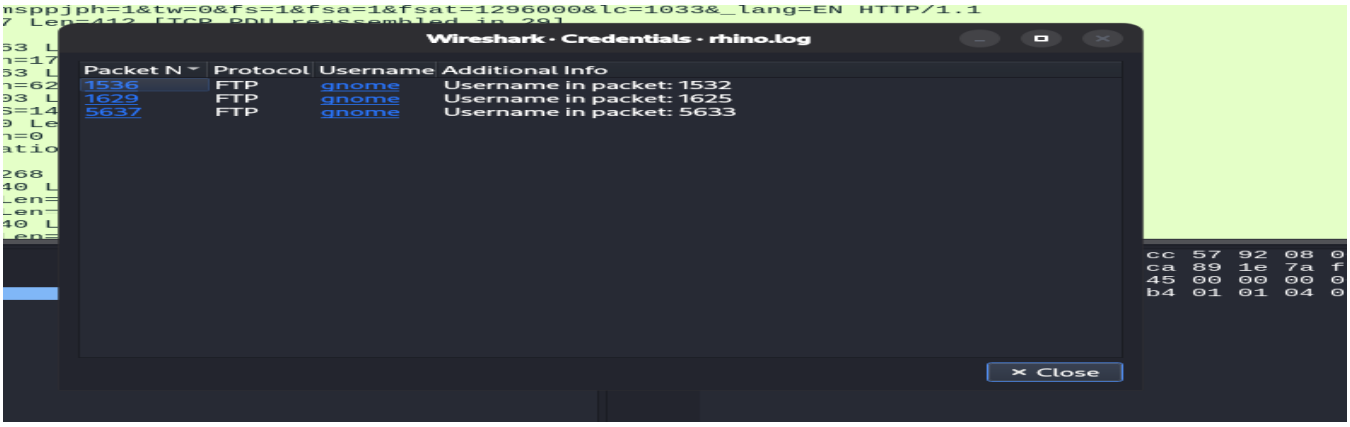
Investigate rhino.log (FTP traffic forensics)

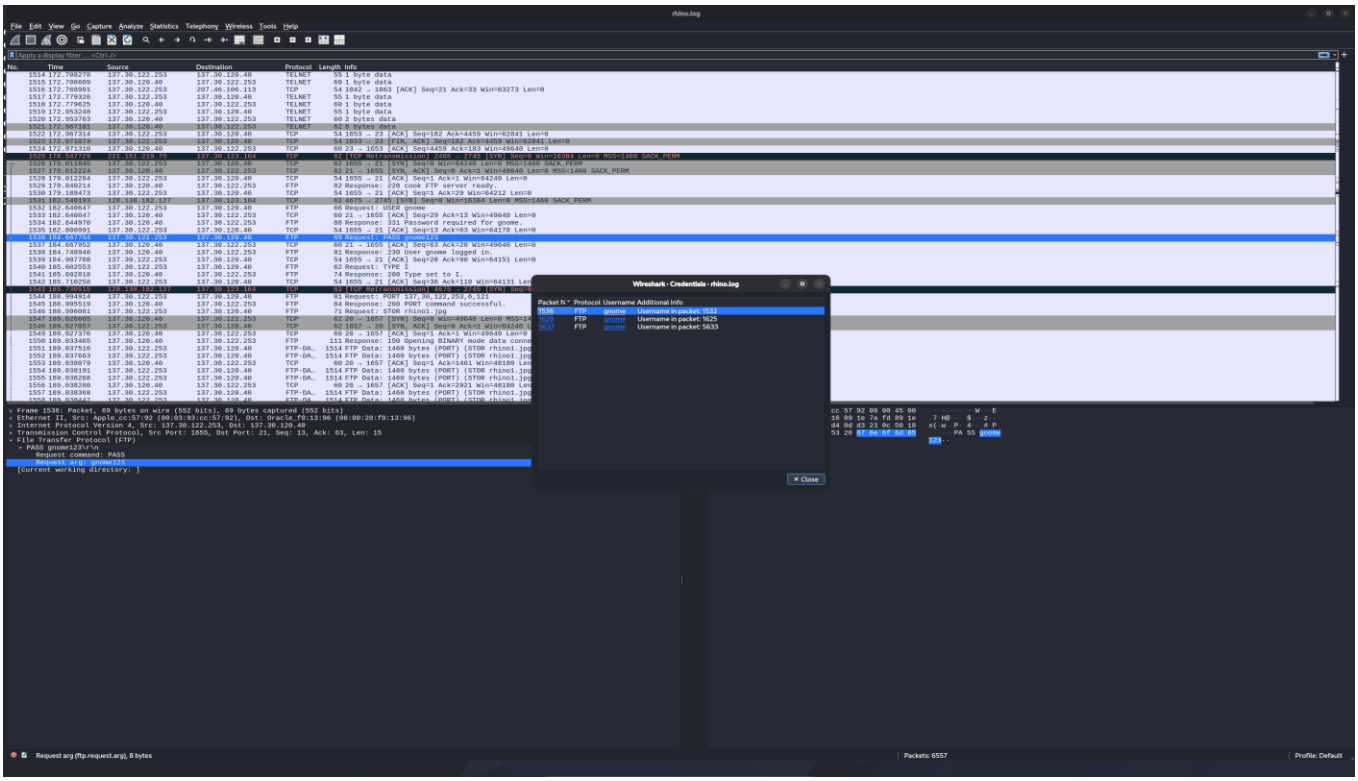


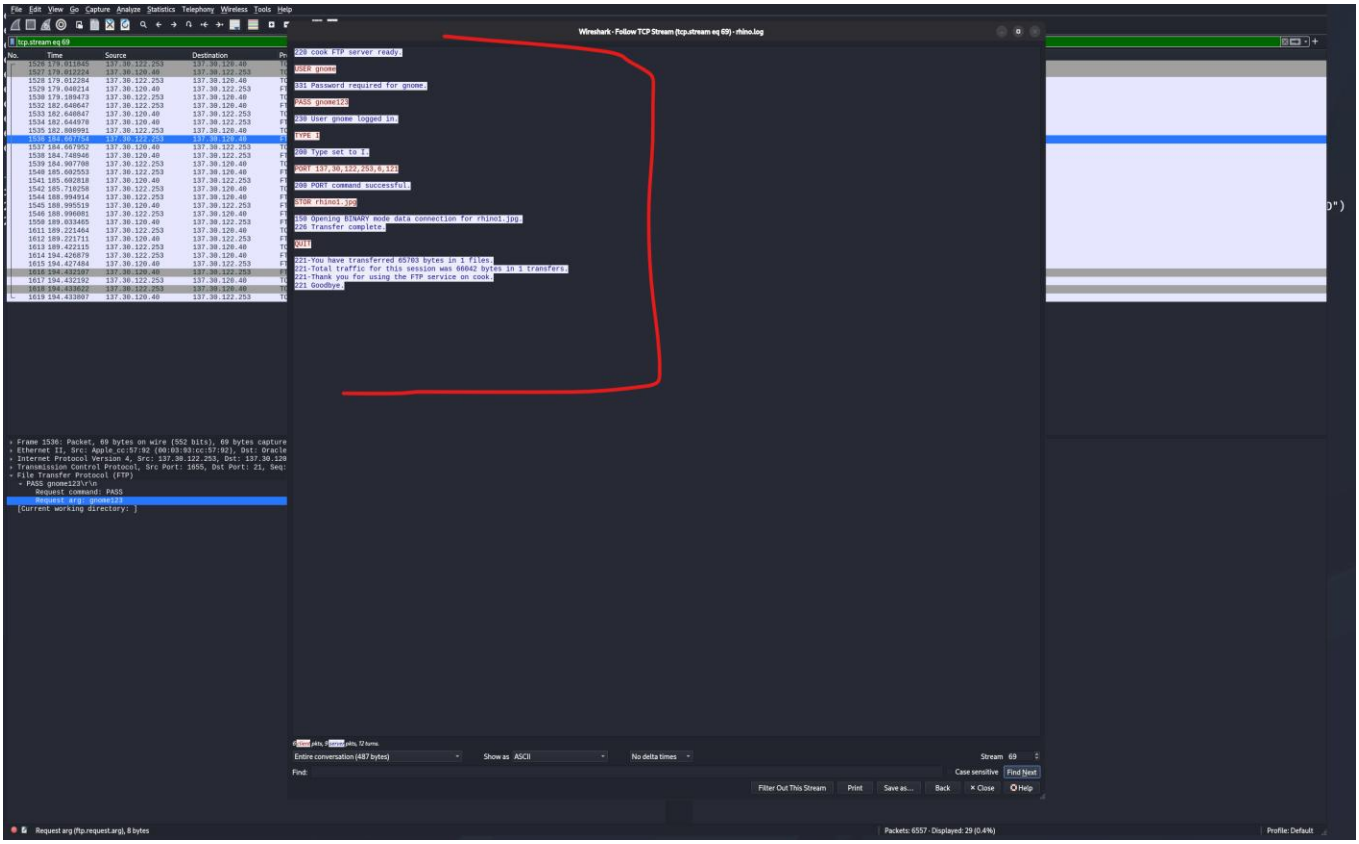
Open rhino.log with Wireshark

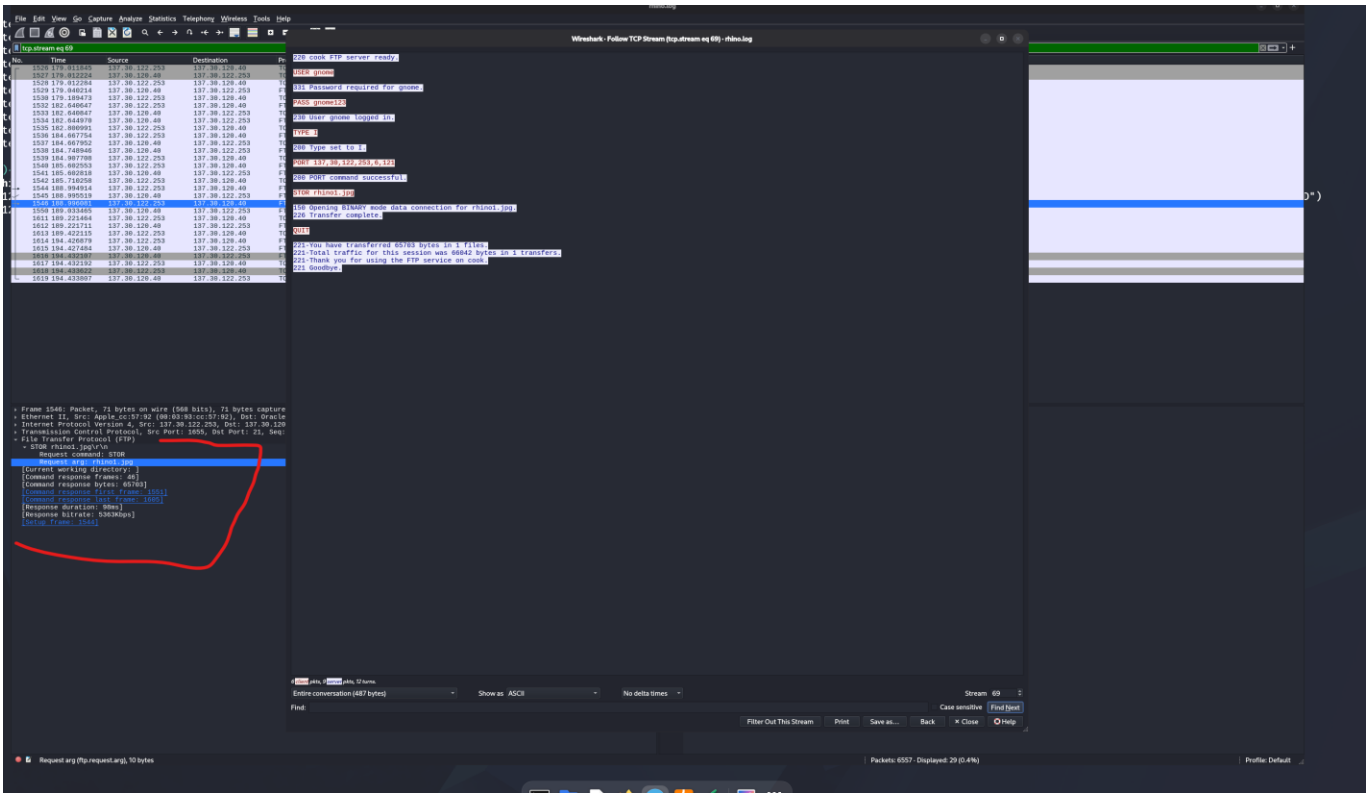


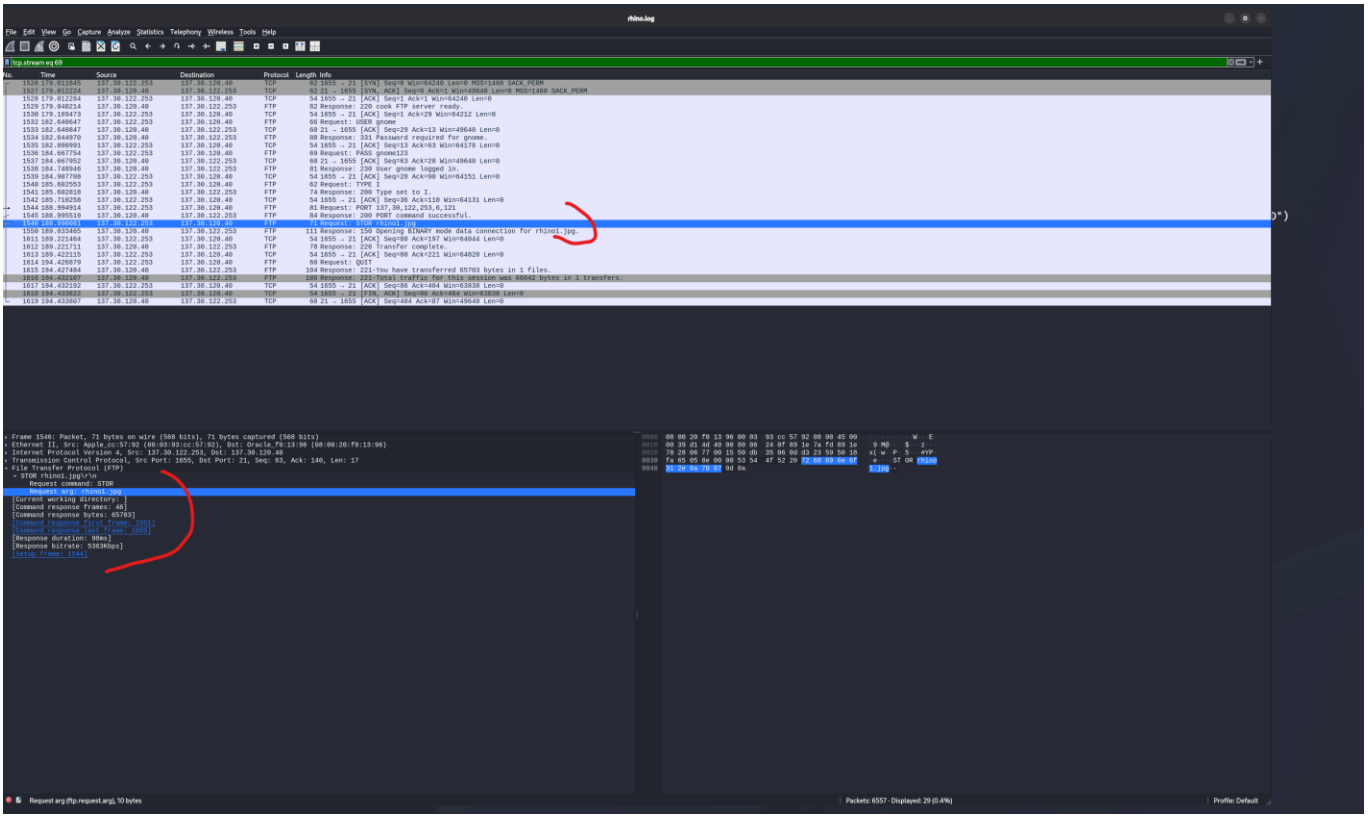
Check credentials (in case users using FTP, which is not encrypted)




rhino.log
Wireshark · Credentials · rhino.log
Packet N ° Protocol Username Additional Info
1536 FTP gnome Username in packet: 1532
1625 FTP gnome Username in packet: 1625
5633 FTP gnome Username in packet: 5633
Close
Request arg (ftp.request.arg), 8 bytes
Packets: 6557
Profile: Default

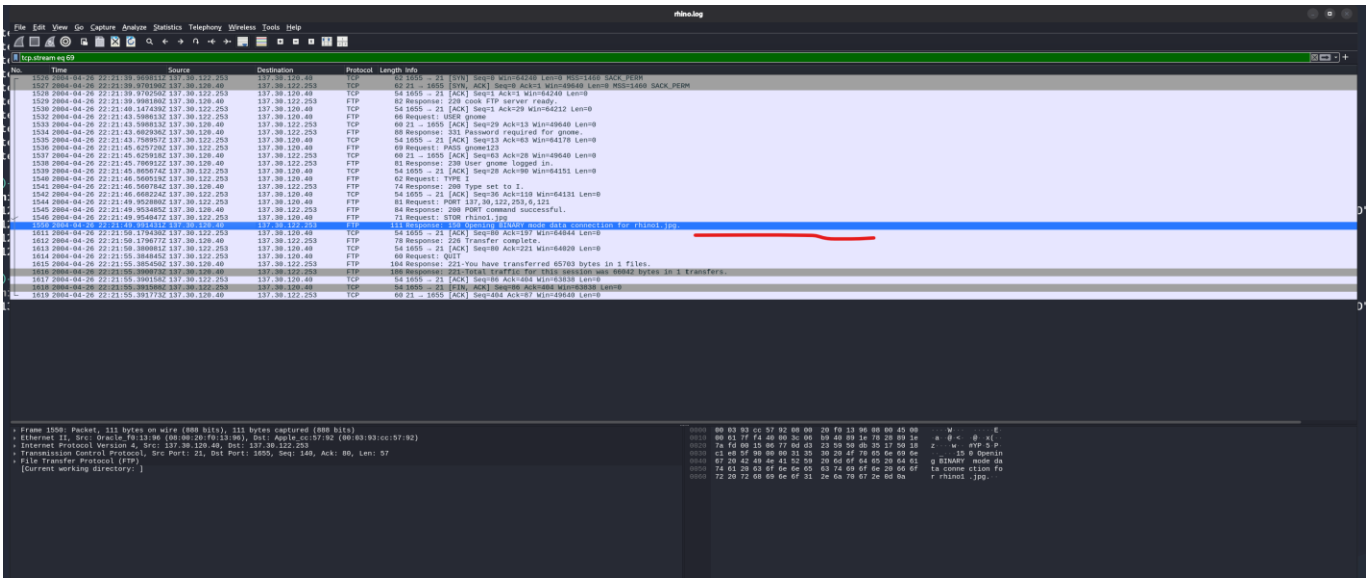

Wireshark · Follow TCP Stream (tcp.stream eq 69) · rhino.log
220 cook FTP server ready.
USER gnome
331 Password required for gnome.
PASS gnome123
230 User gnome logged in.
TYPE I
200 Type set to I.
PORT 137,30,122,253,0,171
200 PORT command successful.
STOR rhino1.jpg
150 Opening BINARY mode data connection for rhino1.jpg.
226 Transfer complete.
QUIT
221-You have transferred 65703 bytes in 1 files.
221-Total traffic for this session was 66642 bytes in 1 transfers.
221-Thank you for using the FTP service on cook.
221 Goodbye.
Entire conversation (487 bytes)
Show as ASCII
No delta times
Stream 69
Find:
Case sensitive
Find Next
Filter Out This Stream
Print
Save as...
Back
Close
Help
Request arg (ftp.request.arg), 8 bytes
Packets: 6557 · Displayed: 29 (0.4%)
Profile: Default


Wireshark · Follow TCP Stream (tcp.stream eq 69) · rhino.log
220 cook FTP server ready.
USER gnome
331 Password required for gnome.
PASS gnome123
230 User gnome logged in.
TYPE I
200 Type set to I.
PORT 137,30,122,253,6,121
200 PORT command successful.
STOR rhino1.jpg
150 Opening BINARY mode data connection for rhino1.jpg.
226 Transfer complete.
QUIT
221-You have transferred 65703 bytes in 1 files.
221-Total traffic for this session was 66042 bytes in 1 transfers.
221-Thank you for using the FTP service on cook.
221 Goodbye.
Request arg: rhino1.jpg
[Current working directory:]
[Command response frames: 46]
[Command response bytes: 65703]
[Response duration: 90ms]
[Response bitrate: 5363Kbps]
Entire conversation (487 bytes)
Show as ASCII
No delta times
Stream 69
Find:
Case sensitive
Find next
Filter Out This Stream
Print
Save as...
Back
Close
Help
Request arg (ftp.request.arg), 10 bytes
Packets: 6557 · Displayed: 29 (0.4%)
Profile: Default

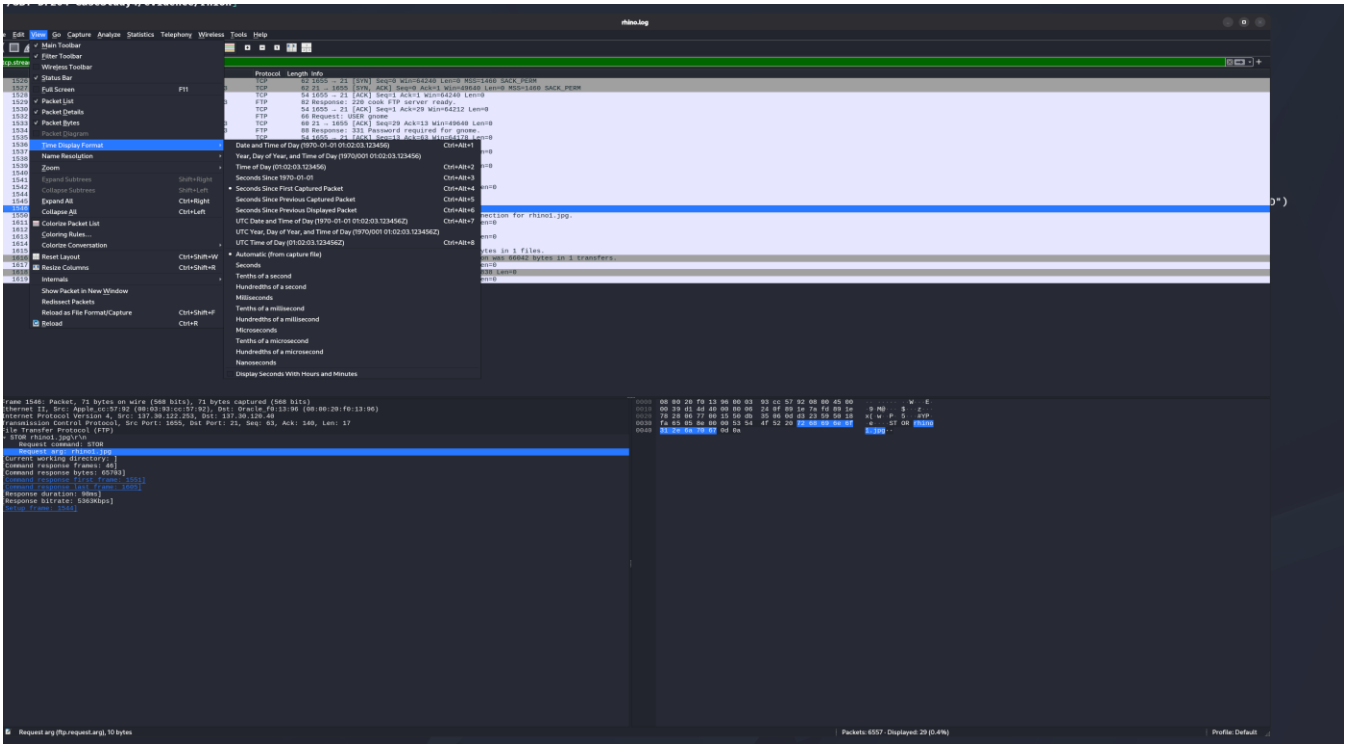

rhino.log
tcp.stream eq 69
Request arg: rhino1.jpg
[Current working directory:]
[Command response frames: 46]
[Command response bytes: 65703]
[Response duration: 90ms]
[Response bitrate: 5363Kbps]
Request arg (ftp.request.arg), 10 bytes
Packets: 6557 · Displayed: 29 (0.4%)
Profile: Default

Open rhino.log with Wireshark

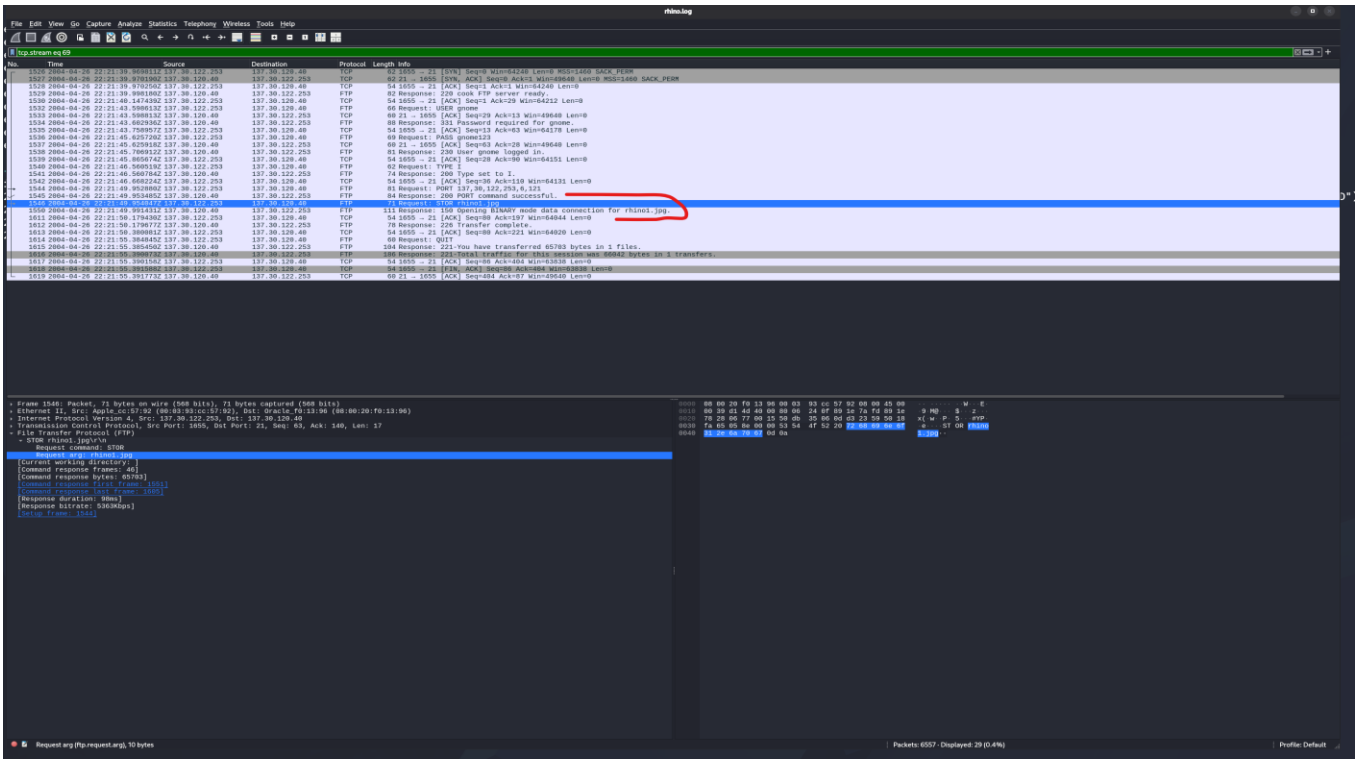
Then type tcp.stream eq 69



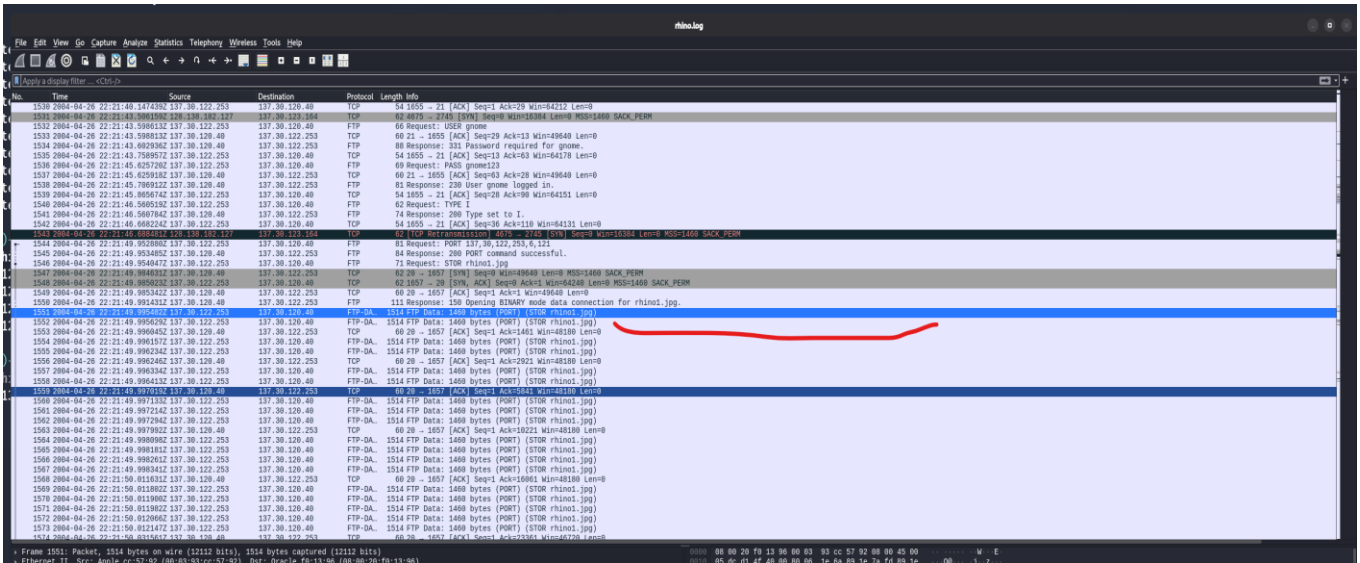
Change time display format

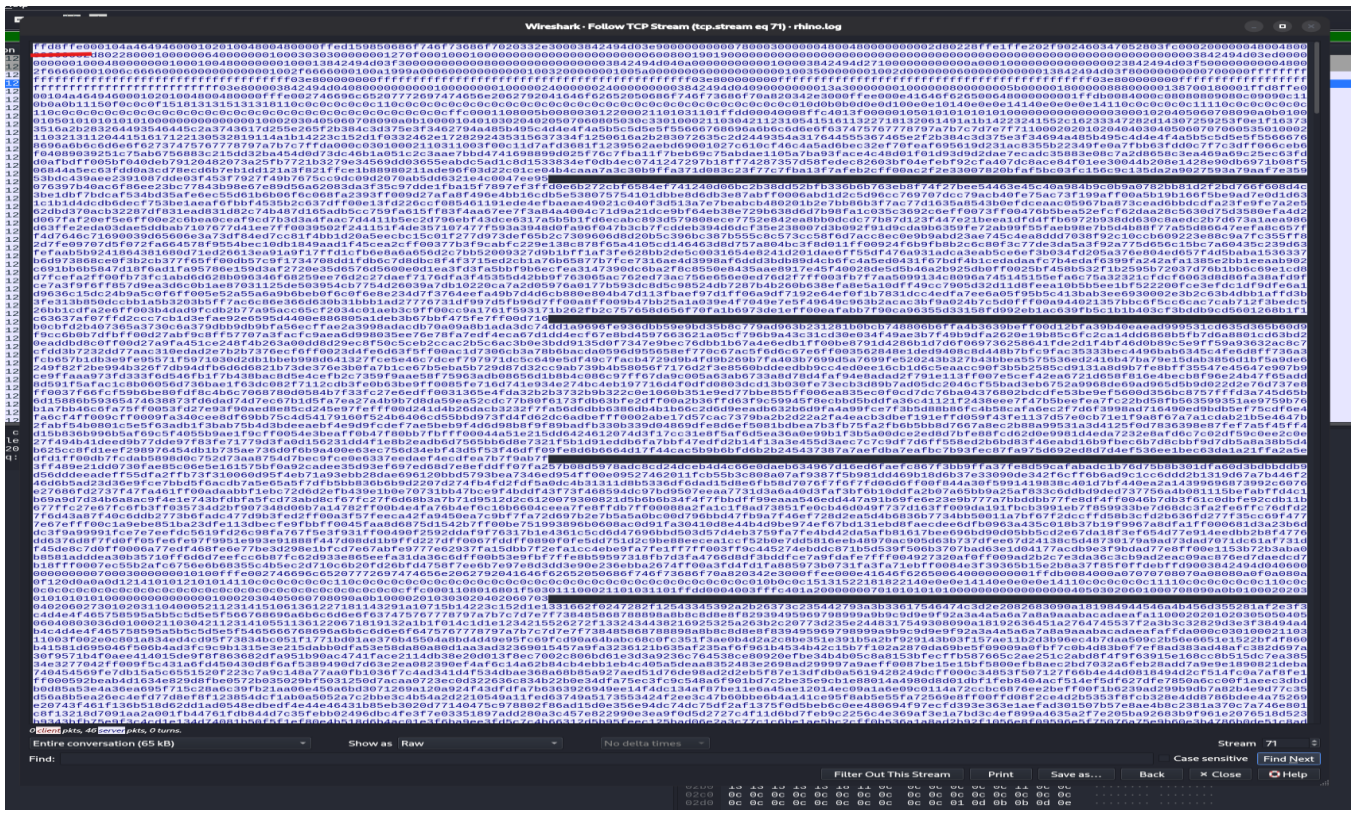


Show the UTC time (2004-04-20 22:21) that suspect starts to transfer photo

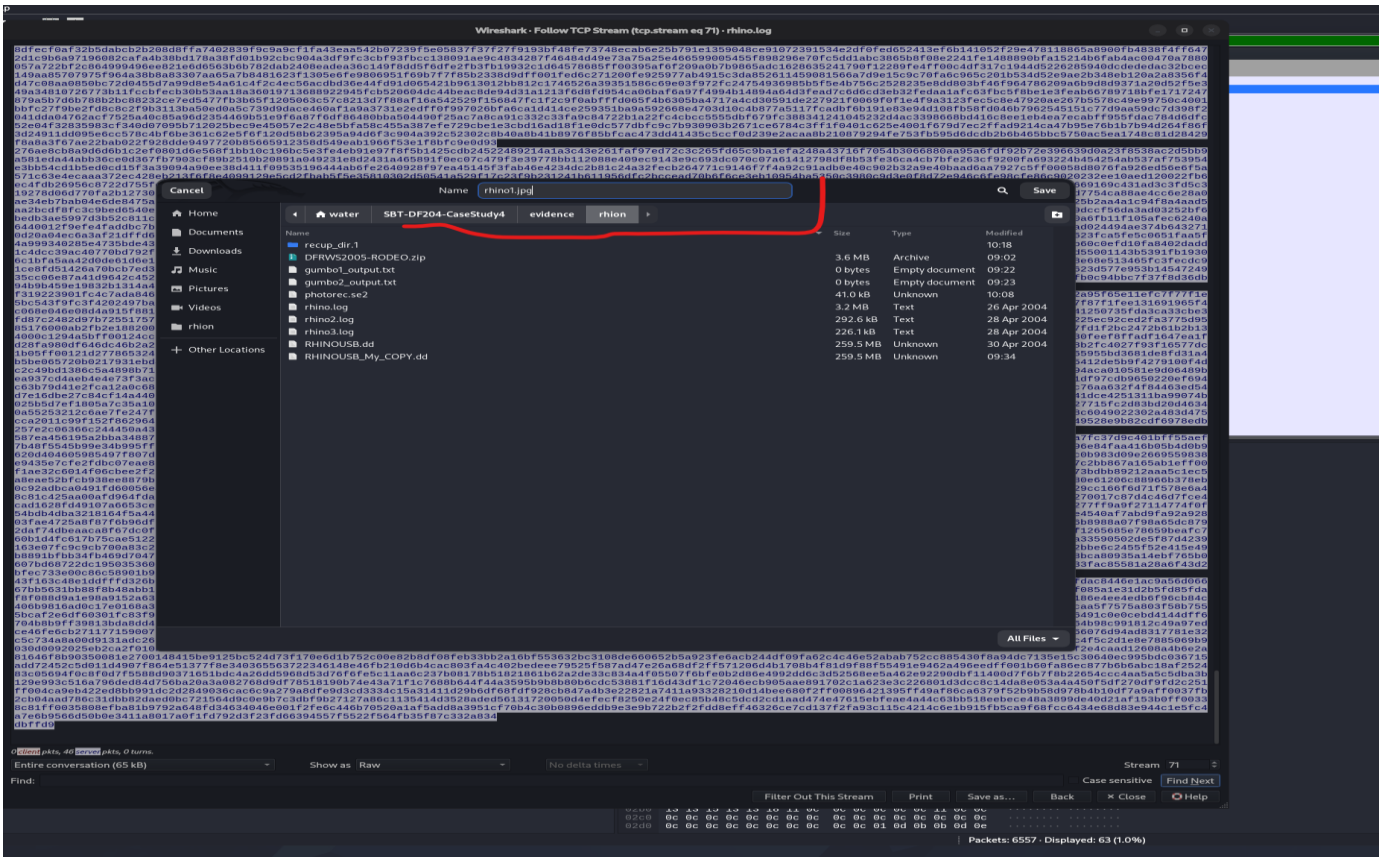


Follow the next packet (1550+1, server to suspect's PC) to collect all packets

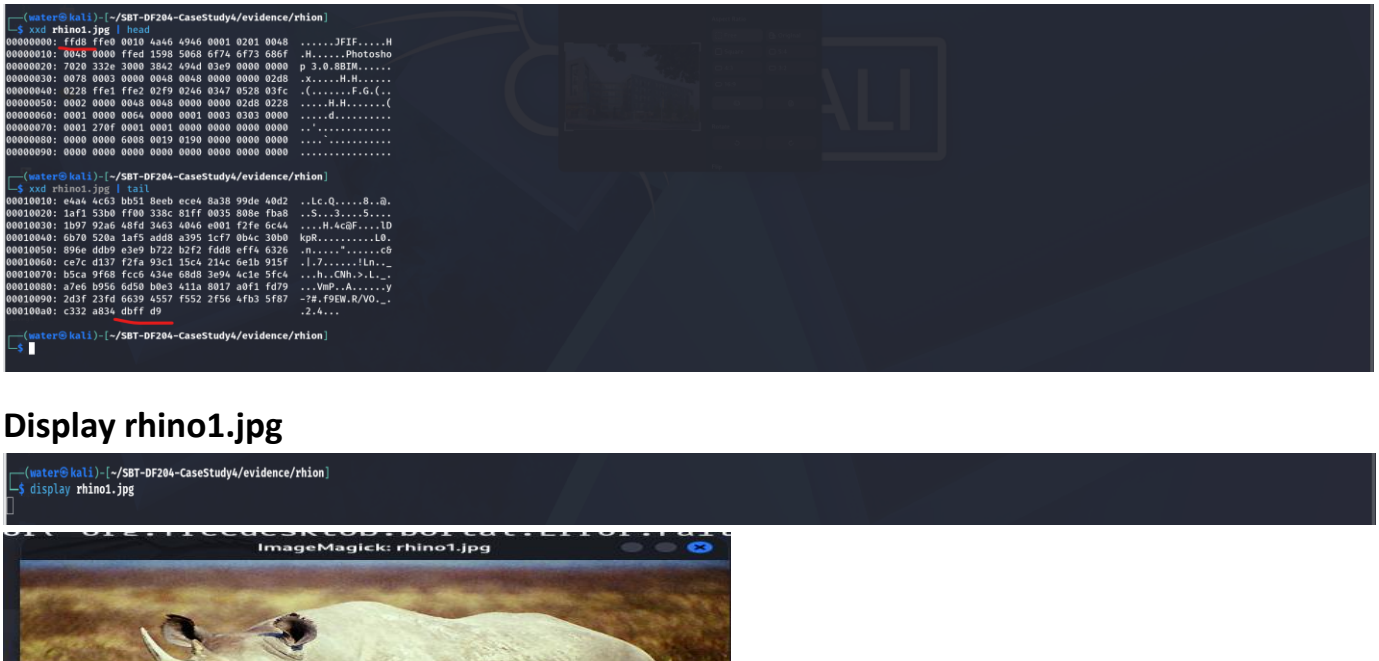



Wireshark · Follow TCP Stream (tcp.stream eq 71) · rhino.log
Entire conversation (65 kB)
Show as Raw
No delta times
Stream 71
Find:
Case sensitive
Find Next
Filter Out This Stream
Print
Save as...
Back
Close
Help


rhino.log
Wireshark · Follow TCP Stream (tcp.stream eq 71) · rhino.log
Entire conversation (65 kB)
Show as Raw
No delta times
Stream 71
Find:
Case sensitive
Find Next
Filter Out This Stream
Print
Save as...
Back
Close
Help
Packets: 6557 · Displayed: 63 (1.0%)



Verify rhino1.jpg magic number

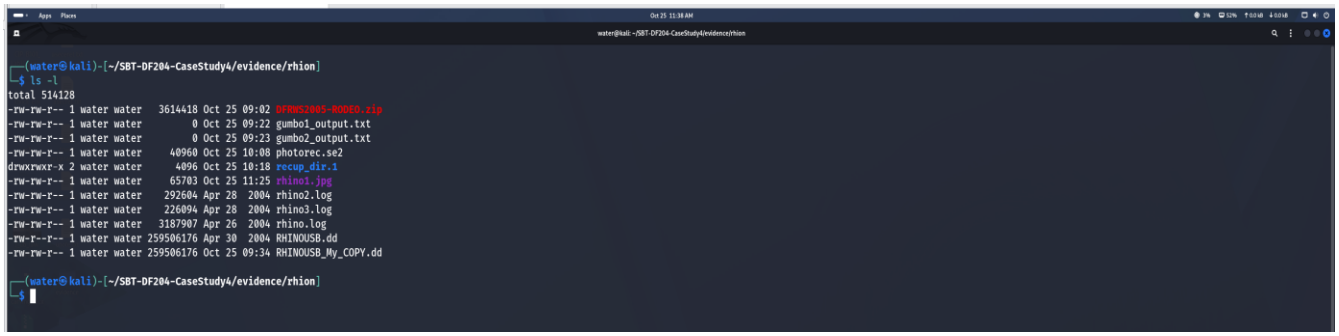


Display rhino1.jpg

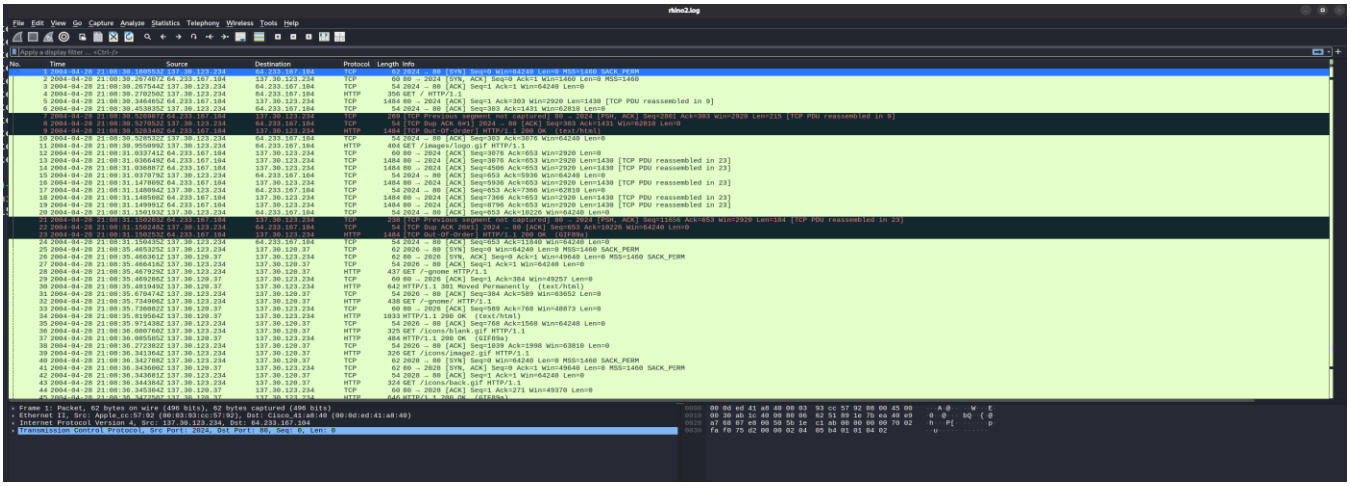


Traffic Log Analysis: HTTP

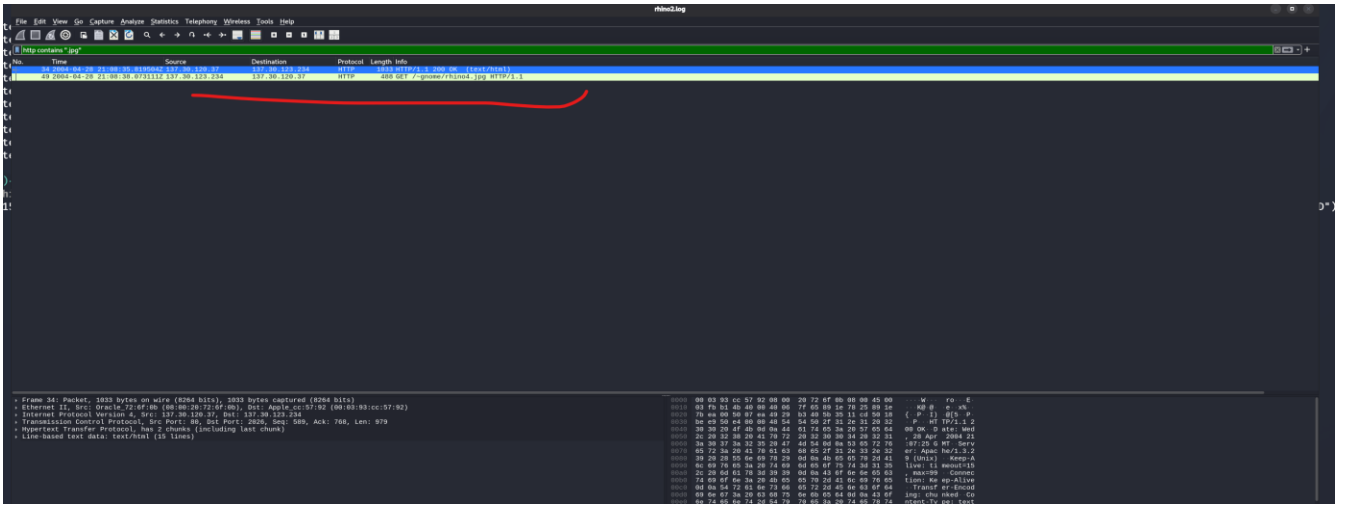
Investigate rhino2.log HTTP traffic forensics



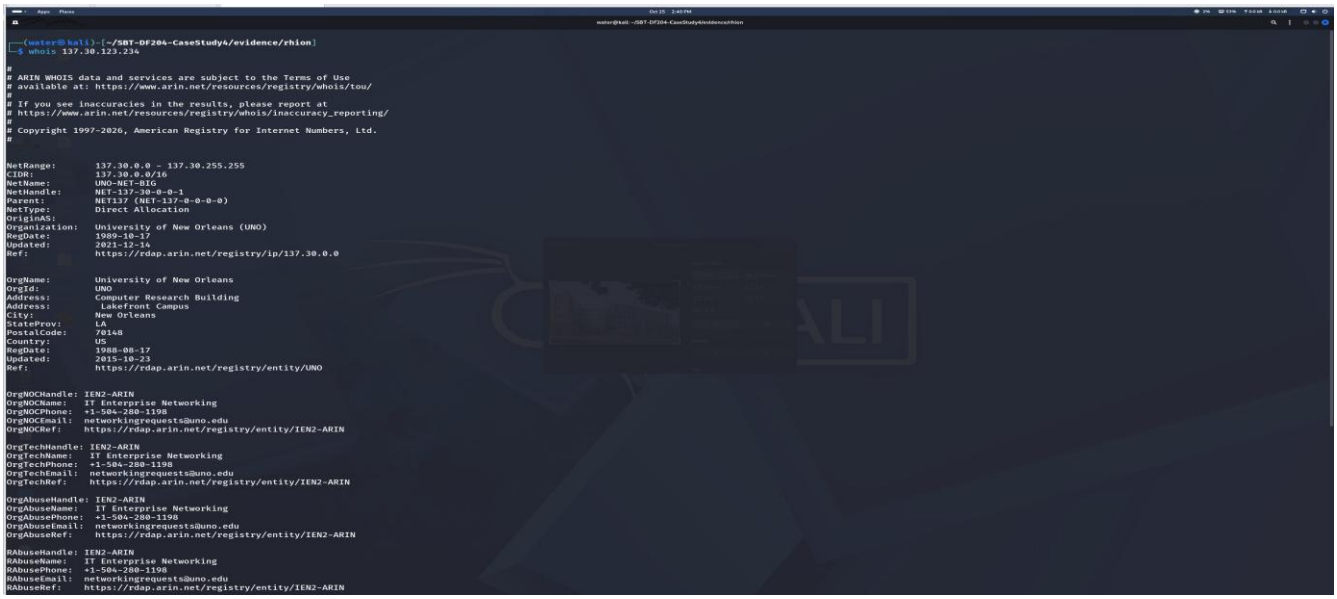




Search suspect's http requests to download .jpg (http contains ".jpg")

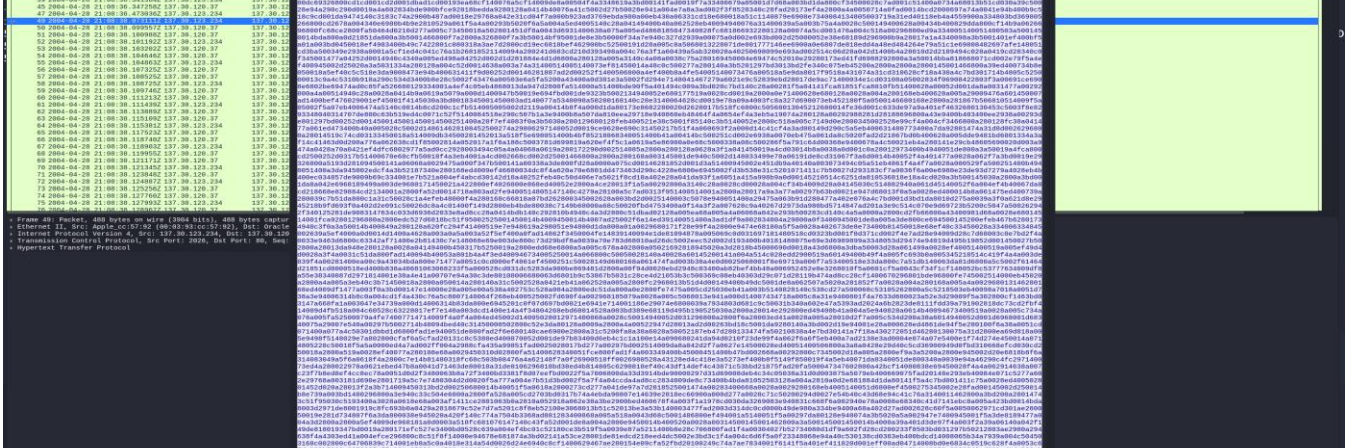


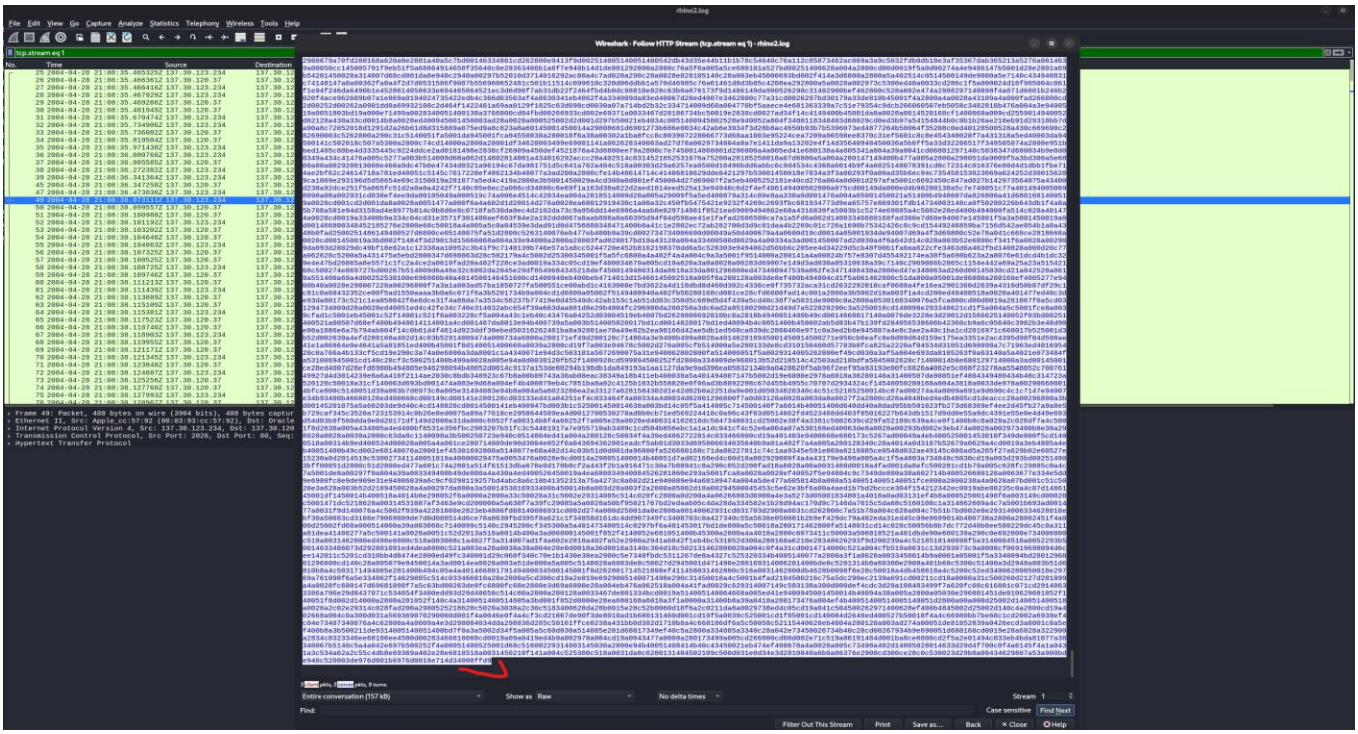
Find who owns the IP



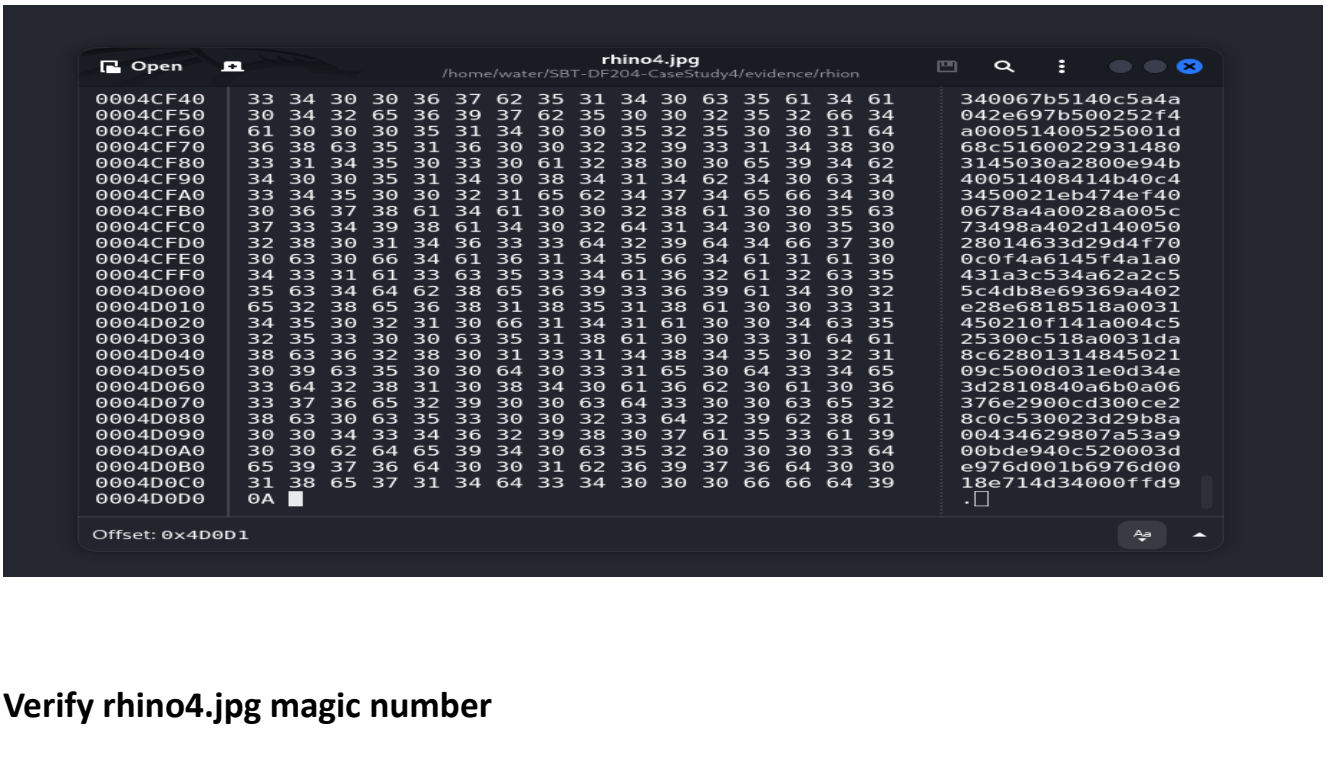


Follow rhino4.jpg HTTP traffic





Save it as rhino4.jpg



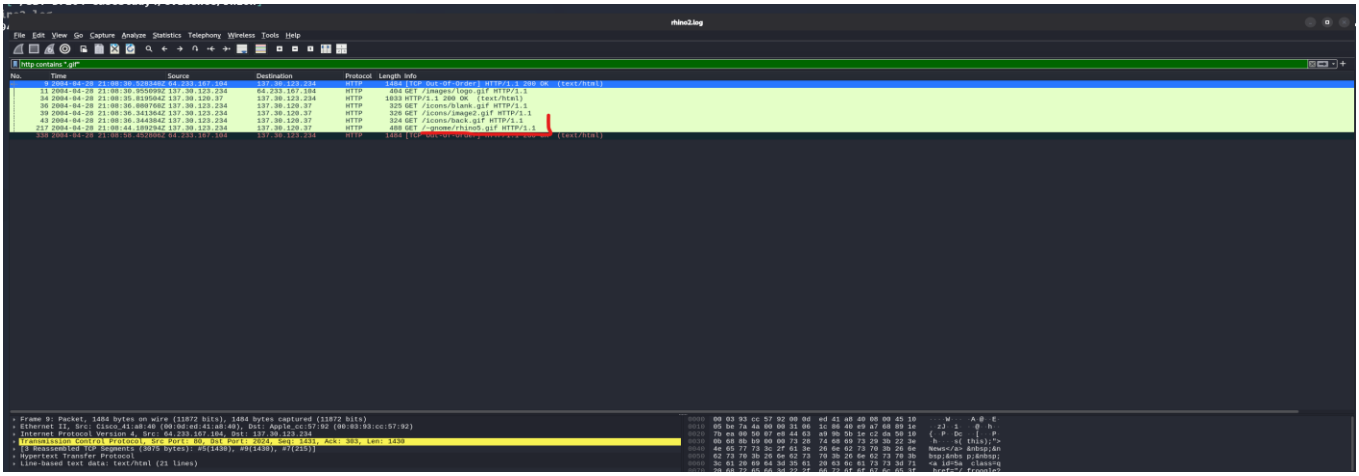
Verify rhino4.jpg magic number

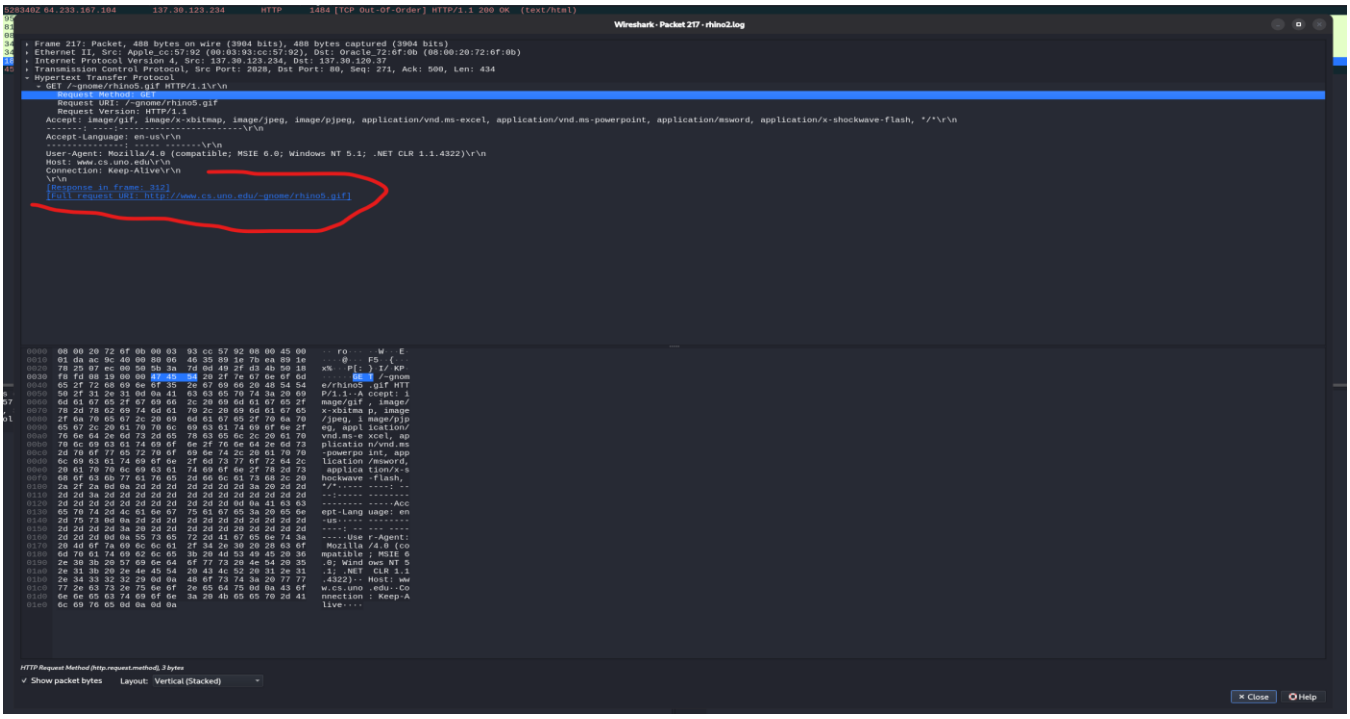
```
┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$ xxd rhino4.jpg | tail
000267d0: 4006 78a4 a002 8a00 5c73 498a 402d 1400  @.x.....\sI.@-..
000267e0: 5028 0146 33d2 9d4f 700c 0f4a 6145 f4a1  P(.F3..Op..JaE..
000267f0: a043 1a3c 534a 62a2 c55c 4db8 e693 69a4  .C.<SJb..\M...i.
00026800: 02e2 8e68 1851 8a00 3145 0210 f141 a004  ...h.Q..1E...A..
00026810: c525 300c 518a 0031 da8c 6280 1314 8450  .%0.Q..1..b....P
00026820: 2109 c500 d031 e0d3 4e3d 2810 840a 6b0a  !....1..N=(...k.
00026830: 0637 6e29 00cd 300c e28c 0c53 0023 d29b  .7n)..0....S.#..
00026840: 8a00 4346 2980 7a53 a900 bde9 40c5 2000  ..CF).zS....@. .
00026850: 3de9 76d0 01b6 976d 0018 e714 d340 00ff  =.v....m.....@..
00026860: d9                                       .

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$ xxd rhino4.jpg | head
00000000: 4745 5420 2f7e 676e 6f6d 6520 4854 5450  GET /~gnome HTTP
00000010: 2f31 2e31 0d0a 4163 6365 7074 3a20 696d  /1.1..Accept: im
00000020: 6167 652f 6769 662c 2069 6d61 6765 2f78  age/gif, image/x
00000030: 2d78 6269 746d 6170 2c20 696d 6167 652f  -xbitmap, image/
00000040: 6a70 6567 2c20 696d 6167 652f 706a 7065  jpeg, image/pjpe
00000050: 672c 2061 7070 6c69 6361 7469 6f6e 2f76  g, application/v
00000060: 6e64 2e6d 732d 6578 6365 6c2c 2061 7070  nd.ms-excel, app
00000070: 6c69 6361 7469 6f6e 2f76 6e64 2e6d 732d  lication/vnd.ms-
00000080: 706f 7765 7270 6f69 6e74 2c20 6170 706c  powerpoint, appl
00000090: 6963 6174 696f 6e2f 6d73 7offf 7264 2c20  ication/msword,

┌──(water㉿kali)-[~/SBT-DF204-CaseStudy4/evidence/rhion]
└─$
```

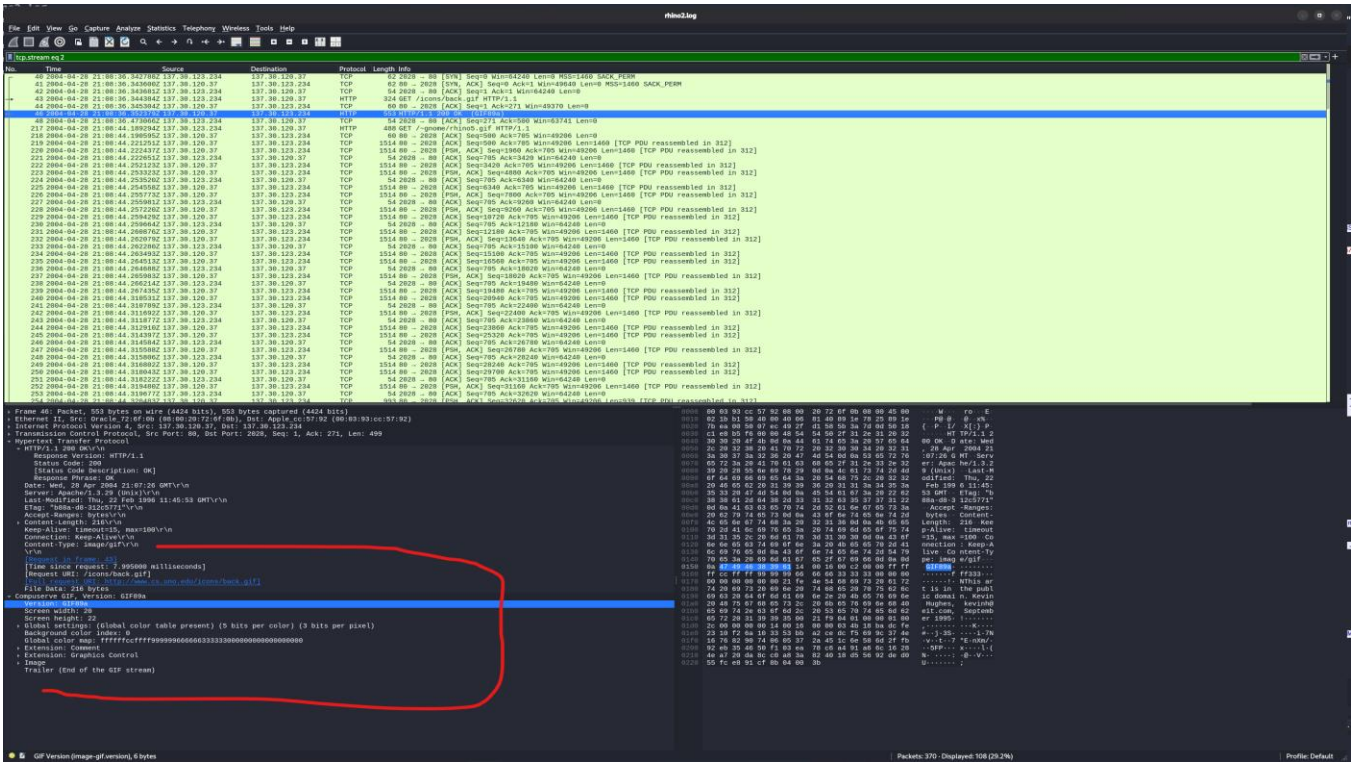
Search .gif HTTP traffic



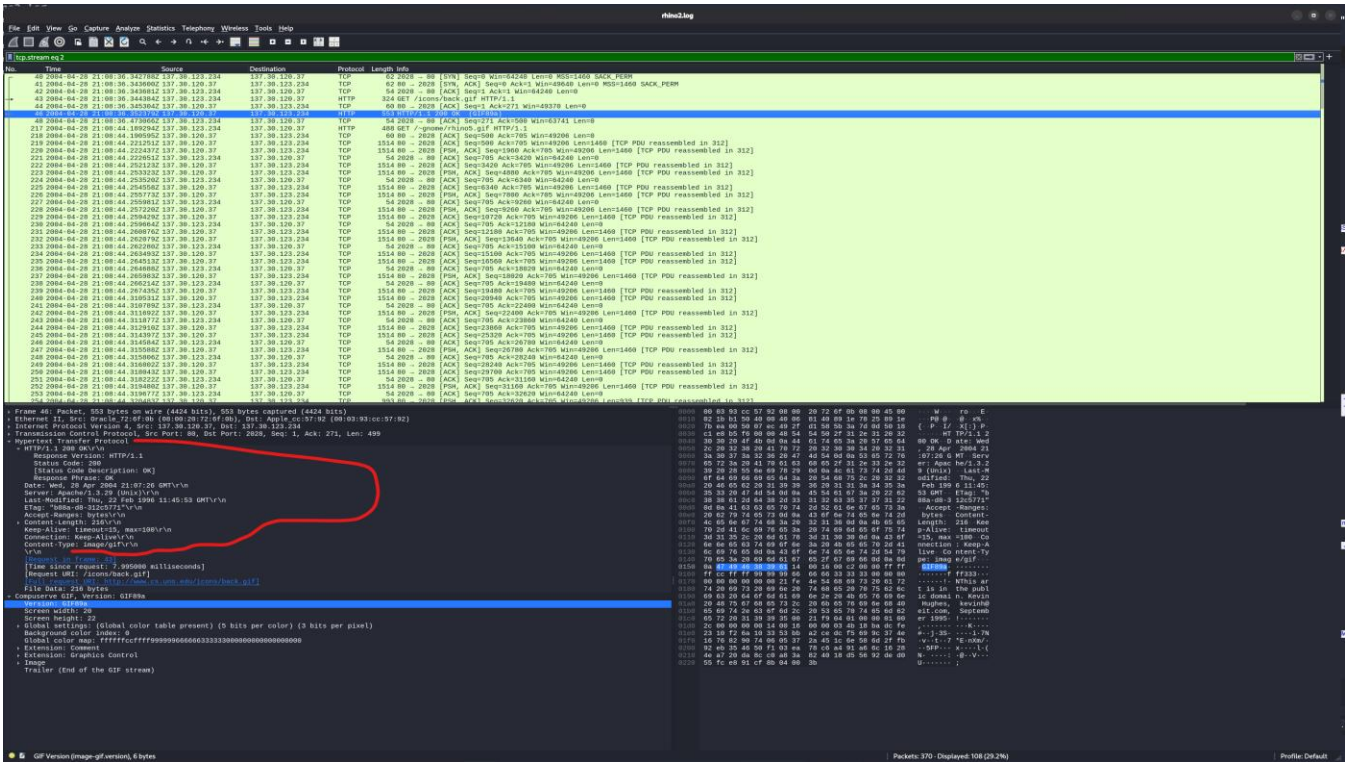


Finding the packet that contains rhino5.gif





Details of how HTTP transfers images



Below is the analysis and result to the eight questions asked on the case study and also relating to all the screenshot above

How did you preserve the case materials? For each supplied disk image and capture, record source, original file name, byte size, SHA-256 value, working-copy method, tool version, and file-access precautions.

I preserved the case materials by keeping the original evidence files unchanged and creating separate working copies for forensic analysis in Kali Linux. The supplied materials included the USB disk image (RHINOUSB.dd), the FTP capture (rhino.log), the HTTP captures (rhino2.log and rhino3.log), and supporting files such as basic.log, image2.log, and the reconstructed image building.jpg. I used file identification, metadata checks, and cryptographic hashes to help verify the integrity of the evidence. MD5 value is 80348c58eec4c328ef1f7709adc56a54 for RHINOUSB.dd and the same MD5 value for its working copy

What deleted or recoverable files are supported? Describe the recovery method, output folder, recovered-file type, hash, visual or metadata check, and whether files are unique or duplicates. Do not report deck answers as your own findings.

The investigation involved examining the RHINOUSB.dd disk image in Kali Linux to identify deleted files and recoverable image data. The case study procedure describes using PhotoRec to recover files and saving the recovered output in a separate folder for further examination. Recovered files was checked using the file command, visual inspection, metadata examination, and SHA-256 hashing to confirm their file types and document their integrity.

What does the FTP capture establish? Identify relevant sessions, transfer direction, packet or stream references, server response, object boundaries, reconstructed file validation, and timestamps. Treat any visible credential as sensitive evidence; do not reuse it.

The FTP investigation involved examining the supplied rhino.log network capture in Wireshark to identify file transfer activity between an FTP client and server. I inspect TCP stream 69 to examine the relevant packets, and follow the server to client traffic to reconstruct the transferred image. The reconstructed file was checked using its file signature, file type, and SHA-256 hash to establish whether it is a valid image. I also identify 20 April 2004 at approximately 22:21 UTC as the time associated with the start of the image transfer

2.7. Analysis and Findings

The investigation focused on examining the USB disk image, recovering files, and analysing FTP and HTTP network traffic. Forensic tools such as Sleuth Kit, PhotoRec, Wireshark, and SHA-256 hashing were identified for examining and validating the evidence. The investigation also considered whether recovered media contained hidden information and whether transferred files could be reconstructed.

2.8. Challenges and Solutions

The case study questions were not straightforward and were sometimes difficult to understand. I relied mainly on the course material to guide me through most of the questions.
Some tasks required me to review the instructions carefully before understanding what was expected.
I used the examples and recommended tools in the course material to guide my investigation. Where I faced difficulties, I followed the available guidance and recorded my findings based on the evidence obtained.

2.9. Conclusion

The case study provides a practical approach to examining digital evidence from a USB disk image and network captures. It demonstrates the importance of file recovery, network traffic analysis, and evidence validation

2.10. Recommendations

Take an online class to help the student to understand the case study better

2.11. References

Class materials Guide from Founder ICDFA on Basic Networking Skills for Digital Forensics, (Illegal Possession Image)

https://onedrive.live.com/?id=%2Fpersonal%2Fa674908a5e2e2f2a%2FDocuments%2FDocuments%2Fdigital%2Dforensics%2Dlab%2FIllegal%5FPossession%5FImages&listurl=%2Fpersonal%2Fa674908a5e2e2f2a%2FDocuments&ithint=folder&e=pbR4WG&migratedtospo=true&redeem=aHR0cHM6Ly8xZHJ2Lm1zL2YvYy9hNjc0OTA4YTVlMmUyZjJhL0lnQTdkT25OeGFiLVRLczN6VmRDX2NvV0FRUlNiV1BLYTd1U3JiNjgtel8wcDgwP2U9cGJSNFdH&ga=1